

Unit 5. Tree-Based Methods

Business Data Analytics: Introductory Machine Learning

Splits, not coefficients

Linear models fit one hyperplane defined by a coefficient vector. Tree models partition the feature space into rectangular cells defined by a sequence of splits, with a constant prediction per cell. The two geometries fail in different ways.

Notation reused across this unit

A few symbols carry different roles in different sections; each occurrence is locally disambiguated, but the table below records the conventions for non-linear readers.

Symbol	Meaning	Where
α	cost-complexity pruning penalty	§2.3
α	linearity-axiom scalar in the Shapley value	§3.5
α	XGBoost L1 leaf-weight penalty	§5.3
L	left child of an internal split	§2, throughout
L (subscript)	parent leaf in the closed-form gain decomposition	§5.2
ν	shrinkage / learning rate (body math)	§4.3, §5
η	row-subsample fraction (body math)	§4.3
eta	XGBoost API parameter for shrinkage ν	§5.3, §6

1 Recursive Partitioning Foundations

This section fixes the tree function class: axis-aligned partitions of the feature space, with a constant prediction in each cell.

1.1 Trees as axis-aligned partitions

A decision tree encodes a recursive partition of the feature space and a constant prediction in each cell. The feature space is $\mathcal{X} = \mathbb{R}^p$ throughout the unit; the prediction space $\mathcal{Y}$ is $\mathbb{R}$ for regression and $\{1, \dots, K\}$ (or, for probability outputs, the K -simplex) for K -class classification.

Definition 5.1 (Binary decision tree). A **binary decision tree** is a finite rooted tree T in which every node has either zero children (a **leaf**) or exactly two (an **internal node**); the unique node with no parent is the **root**. The tree carries the following labels:

- every internal node t has a **split rule** (j_t, s_t) with $j_t \in \{1, \dots, p\}$ and $s_t \in \mathbb{R}$, routing an input $x \in \mathcal{X}$ to the **left** child when $x_{j_t} \leq s_t$ and to the **right** child otherwise;
- the leaves are enumerated $m = 1, \dots, M$ where $M = |T|$, and each leaf m has a **leaf value** $c_m \in \mathcal{Y}$.

Definition 5.2 (Region induced by a leaf). For each leaf m , let $\mathcal{R}_m \subseteq \mathcal{X}$ be the set of inputs that reach m when the split rules of Definition 5.1 are applied from the root. Each $\mathcal{R}_m$ is an axis-aligned rectangle (a finite intersection of half-spaces $\{x : x_j \leq s\}$ and $\{x : x_j > s\}$); the

family $\{\mathcal{R}_m\}_{m=1}^M$ partitions $\mathcal{X}$.

Definition 5.3 (Tree predictor). The **predictor** encoded by a tree T with leaf values $\{c_m\}_{m=1}^M$ is the piecewise-constant function

$$f_T(x) = \sum_{m=1}^M c_m \mathbb{1}\{x \in \mathcal{R}_m\}, \quad x \in \mathcal{X}.$$

The **tree class** on $\mathcal{X}$ is

$$\mathcal{F}_{\text{tree}} = \{f_T : T \text{ is a binary decision tree on } \mathcal{X}\}.$$

Three structural facts will recur:

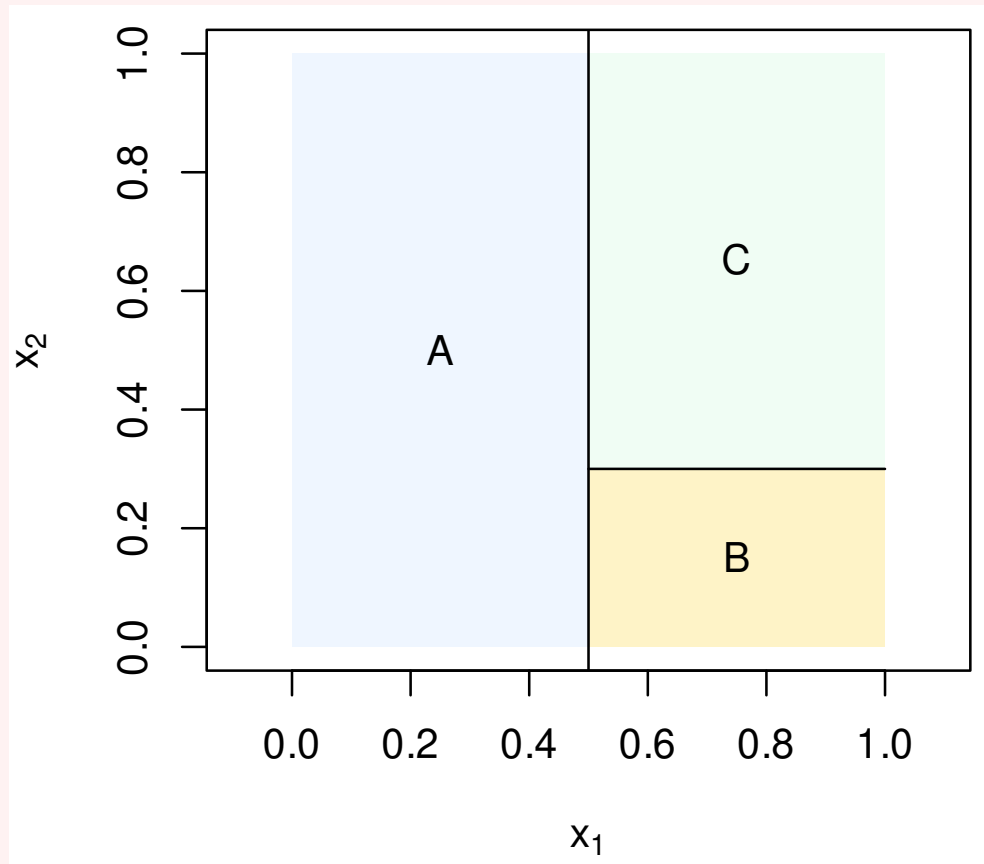
- The partition $\{\mathcal{R}_m\}$ is determined by the splits alone; the leaf values c_m play no role in selecting splits.
- Once the partition is fixed, leaf values minimize the loss separately in each cell, in closed form for the losses used in this unit.

- $\mathcal{F}_{\text{tree}}$ contains every function that is constant on the cells of some axis-aligned rectangular partition of $\mathcal{X}$. Such piecewise-constant functions are dense in $L^2(P)$ for any P with bounded support, and on a finite sample of size n a tree with n leaves interpolates any labels exactly. An unconstrained tree class therefore overfits any finite sample, and some form of capacity control is mandatory.

Example 5.1 (A two-feature tree and its partition). A three-leaf tree on $[0, 1]^2$ with split rules

$$\text{root: } x_1 \leq 0.5 \rightarrow A; \quad \text{else: } x_2 \leq 0.3 \rightarrow B \text{ else } C$$

induces the partition $\mathcal{R}_A = \{x_1 \leq 0.5\}$, $\mathcal{R}_B = \{x_1 > 0.5, x_2 \leq 0.3\}$, $\mathcal{R}_C = \{x_1 > 0.5, x_2 > 0.3\}$. The figure below renders the three regions.



Why axis-aligned

Axis-aligned splits are one modeling choice; oblique trees, which split on linear combinations $\beta^\top x \leq s$, also fit Definition 5.1. The axis-aligned restriction has two practical consequences. First, the best-split search at a node decomposes into p separate one-dimensional sweeps, which keeps greedy growing tractable for p in the hundreds. Second, the predictor is invariant to monotone feature transformations: training on $\phi(x_j)$ in place of x_j for any monotone increasing ϕ yields the same partition and predictions. This invariance is why tree models skip the standardization step that ridge, lasso (Unit 2 §2.7.1), logistic regression, and SVM require.

1.2 Empirical risk on a partition

Fix a tree T with regions $\{\mathcal{R}_m\}_{m=1}^M$ from Definition 5.2. Given a loss $\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}_{\geq 0}$ (Unit 1 Definition 1.17) and training rows $\{(x_i, y_i)\}_{i=1}^n$, write

$$I_m := \{i : x_i \in \mathcal{R}_m\}, \quad n_m := |I_m|$$

for the index set and count of training rows that reach leaf m .

Lemma 5.4 (Per-leaf decomposition of empirical risk). The empirical risk of the tree predictor f_T (Definition 5.3) on the training sample is

$$\widehat{\mathcal{R}}_n(f_T) = \frac{1}{n} \sum_{m=1}^M \sum_{i \in I_m} \ell(y_i, c_m).$$

The leaf values $\{c_m\}_{m=1}^M$ separate: minimizing $\widehat{\mathcal{R}}_n(f_T)$ over the leaf values reduces to

$$c_m^* \in \arg \min_{c \in \mathcal{Y}} \sum_{i \in I_m} \ell(y_i, c)$$

solved independently for each m .

Proof. By Definition 5.3, $f_T(x_i) = c_m$ for the unique leaf m with $x_i \in \mathcal{R}_m$ (the regions partition

$\mathcal{X}$, Definition 5.2). Each training index $i \in \{1, \dots, n\}$ therefore lies in exactly one I_m , so

$$\widehat{\mathcal{R}}_n(f_T) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, f_T(x_i)) = \frac{1}{n} \sum_{m=1}^M \sum_{i \in I_m} \ell(y_i, c_m).$$

The double sum has the form $\sum_m g_m(c_m)$ with $g_m(c) := \sum_{i \in I_m} \ell(y_i, c)$; minimization separates over the c_m . $\square$

The per-leaf optimum has a closed form for the three losses used in this unit.

Example 5.2 (Closed-form leaf values). For a fixed partition with leaf row counts $\{n_m\}_{m=1}^M$:

- **Squared loss** ($\mathcal{Y} = \mathbb{R}$, $\ell(y, c) = (y - c)^2$). The per-leaf objective $\sum_{i \in I_m} (y_i - c)^2$ is a strictly convex quadratic in c , minimized at the leaf mean $\bar{y}_m := n_m^{-1} \sum_{i \in I_m} y_i$.
- **0/1 loss** ($\mathcal{Y} = \{1, \dots, K\}$, $\ell(y, c) = \mathbb{1}\{y \neq c\}$). Writing $n_{m,k} := |\{i \in I_m : y_i = k\}|$ for per-leaf class counts, the per-leaf objective is $\sum_{i \in I_m} \mathbb{1}\{y_i \neq c\} = n_m - n_{m,c}$, minimized at the modal class $c_m^* \in \arg \max_k n_{m,k}$ (ties broken arbitrarily).
- **Cross-entropy** ($\mathcal{Y} = \Delta^{K-1}$, the probability simplex; $\ell(y, p) = -\sum_k \mathbb{1}\{y = k\} \log p_k$).

The per-leaf objective $-\sum_k n_{m,k} \log p_k$ is minimized at the empirical class proportions $\hat{p}_{m,k} = n_{m,k}/n_m$.

Proof of the cross-entropy case in Example 5.2. The objective $-\sum_k n_{m,k} \log p_k$ is convex in p on the simplex (taking $0 \log 0 = 0$). The Lagrangian for $\sum_k p_k = 1$ is

$$\mathcal{L}(p, \mu) = -\sum_k n_{m,k} \log p_k + \mu \left(\sum_k p_k - 1 \right),$$

and stationarity gives $-n_{m,k}/p_k + \mu = 0$, hence $p_k = n_{m,k}/\mu$. The simplex constraint fixes $\mu = n_m$, yielding $p_k^* = n_{m,k}/n_m$. $\square$

Lemma 5.4 and Example 5.2 reduce tree learning to choosing the partition; leaf values then follow in closed form. The remaining problem,

$$\min_T \frac{1}{n} \sum_{m=1}^{|T|} \min_{c \in \mathcal{Y}} \sum_{i \in I_m} \ell(y_i, c),$$

is over the combinatorial set of binary trees on $\mathcal{X}$.

1.3 Why greedy splits

The minimization at the end of §5.1.2 has no efficient exact solver: even “find the smallest tree that fits” is NP-complete.

Theorem 5.5 (Hyafil and Rivest, 1976). The decision problem MIN-TREE is NP-complete:

- **INPUT.** A finite training sample $\{(x_i, y_i)\}_{i=1}^n \subset \{0, 1\}^p \times \{0, 1\}$ and a positive integer K .
- **QUESTION.** Does there exist a binary decision tree with at most K leaves that correctly classifies every training row?

Proof. In NP. Given a candidate tree T , verify in time $O(K \cdot n \cdot p)$ that T has at most K leaves and classifies every training row correctly.

NP-hard via Set Cover. SET-COVER is NP-complete (Karp, 1972; Hyafil and Rivest’s original reduction used Exact Cover by 3-Sets):

- **INPUT.** A universe $U = \{u_1, \dots, u_q\}$, a sub-collection $\mathcal{S} = \{S_1, \dots, S_m\}$ of subsets of U with $\bigcup_j S_j = U$, and a positive integer k .
- **QUESTION.** Does there exist $\mathcal{S}' \subseteq \mathcal{S}$ with $|\mathcal{S}'| \leq k$ such that $\bigcup_{S \in \mathcal{S}'} S = U$?

Construction. From $(U, \mathcal{S}, k)$, build a MIN-TREE instance with $p = m$ features (one per set),

$n = q + 1$ rows, and bound $K = k + 1$:

- For each $u \in U$, one positive row $a_u \in \{0, 1\}^m$ with $(a_u)_j = 1$ iff $u \in S_j$.
- One negative row $b = (0, \dots, 0)$.

The construction is polynomial in the input size.

Cover $\Rightarrow$ tree. Suppose $\mathcal{S}' = \{S_{j_1}, \dots, S_{j_{k'}}\}$ covers U with $k' \leq k$. Build a **comb tree**: along the leftmost path, test features $j_1, \dots, j_{k'}$ in order; the right child of each test is a positive leaf, and the left child after $j_{k'}$ is a negative leaf. Leaf count: $k' + 1 \leq K$.

Every entry of b is 0, so b takes left at every test and lands at the negative leaf. For each a_u , let t be the smallest index in $\{1, \dots, k'\}$ with $u \in S_{j_t}$ (such t exists because $\mathcal{S}'$ covers U); then $(a_u)_{j_s} = 0$ for $s < t$ and $(a_u)_{j_t} = 1$, so a_u takes left at $j_1, \dots, j_{t-1}$, right at j_t , and lands at a positive leaf.

Tree $\Rightarrow$ cover. Suppose T has $|T| \leq K = k + 1$ leaves and classifies every training row correctly. Let $j_1, \dots, j_t$ be the features tested along b 's root-to-leaf path. Every $b_j = 0$, so b takes left at every test; its leaf has label 0.

For each $u \in U$, a_u must reach a positive leaf. From the root, a_u stays with b as long as $u \notin S_{j_s}$ on the tests j_s encountered. If $u \notin S_{j_s}$ for every s , then a_u ends at b 's (negative) leaf, contradicting a_u 's positive label. So some s has $u \in S_{j_s}$, and $\mathcal{S}' := \{S_{j_1}, \dots, S_{j_t}\}$ (de-duplicated) covers U .

Each of the t internal nodes on b 's path has a right child rooting a subtree with at least one leaf; with the leaf at the end of b 's path, $|T| \geq t + 1$. Combined with $|T| \leq k + 1$, $t \leq k$, so $|\mathcal{S}'| \leq t \leq k$. $\square$

Example 5.3 (Reduction in action). Take $U = \{u_1, u_2, u_3\}$ and $\mathcal{S} = \{S_1, S_2, S_3\}$ with $S_1 = \{u_1, u_2\}$, $S_2 = \{u_2, u_3\}$, $S_3 = \{u_1, u_3\}$. The minimum cover has $k = 2$. The reduction sets $K = 3$ and produces

$$a_{u_1} = (1, 0, 1), \quad a_{u_2} = (1, 1, 0), \quad a_{u_3} = (0, 1, 1), \quad b = (0, 0, 0).$$

The cover $\{S_1, S_2\}$ gives the comb tree of Figure 5.1: test f_1 then f_2 down the leftmost path, positive leaves on each right side, negative leaf at the bottom. Three leaves; all four rows classified. No two-leaf tree (one internal test) works: for any feature f_j , the left child of the test on f_j contains b together with the a_{u_i} for which $u_i \notin S_j$.

Example 5.4 (Reverse direction in action). Take $U = \{u_1, u_2, u_3, u_4\}$ and $\mathcal{S} = \{S_1, S_2, S_3, S_4\}$ with $S_1 = \{u_1, u_2\}$, $S_2 = \{u_3, u_4\}$, $S_3 = \{u_1, u_3\}$, $S_4 = \{u_2, u_4\}$. The minimum cover has $k = 2$.

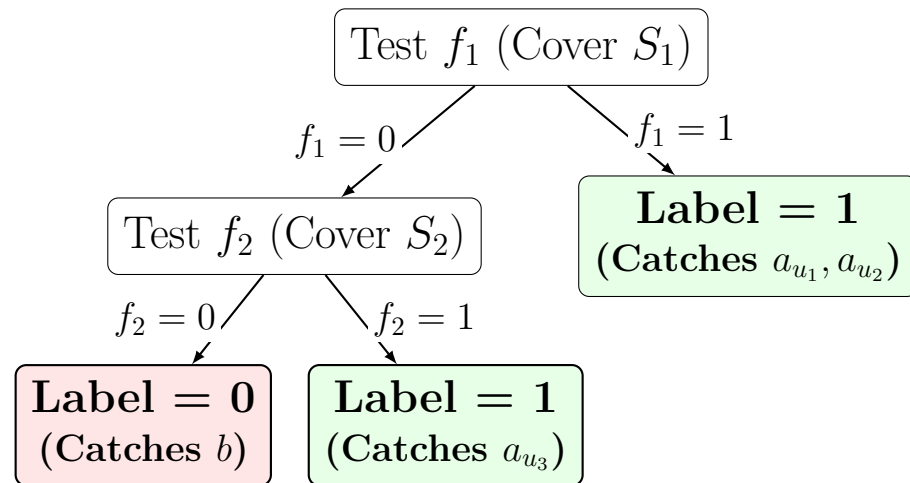


Figure 5.1: Comb tree for the cover $\{S_1, S_2\}$ in Example 5.3.

The reduction sets $K = 3$ and produces

$$a_{u_1} = (1, 0, 1, 0), \quad a_{u_2} = (1, 0, 0, 1), \quad a_{u_3} = (0, 1, 1, 0), \quad a_{u_4} = (0, 1, 0, 1), \quad b = (0, 0, 0, 0).$$

The non-comb 4-leaf tree of Figure 5.2 classifies all five rows correctly. Tracing b : $b_{f_1} = 0$ goes left at the root, $b_{f_2} = 0$ goes left at the next test, lands at the negative leaf. The features along the path give $\mathcal{S}' = \{S_1, S_2\}$, an optimal cover of size 2.

The bound $|T| \geq t + 1$ is loose here ($4 \geq 3$): the right subtree of the root holds two leaves not on b 's path. A comb tree producing the same cover would have $|T| = t + 1 = 3$ leaves.

This restricted version (binary features, exact fit) is intractable; the general empirical-risk-minimization problem is no easier. Tree algorithms therefore use a top-down greedy heuristic.

Definition 5.6 (Top-down greedy growing). **Top-down greedy growing** starts with the training sample at the root, recursively splits each node, and stops when a node-level criterion is met:

- **Split rule.** At a node holding rows $I \subseteq \{1, \dots, n\}$, choose $(j^*, s^*) \in \arg \min_{(j, s)} Q(I; j, s)$, where Q is a per-node objective (e.g., post-split SSE for regression, post-split Gini for classifi-

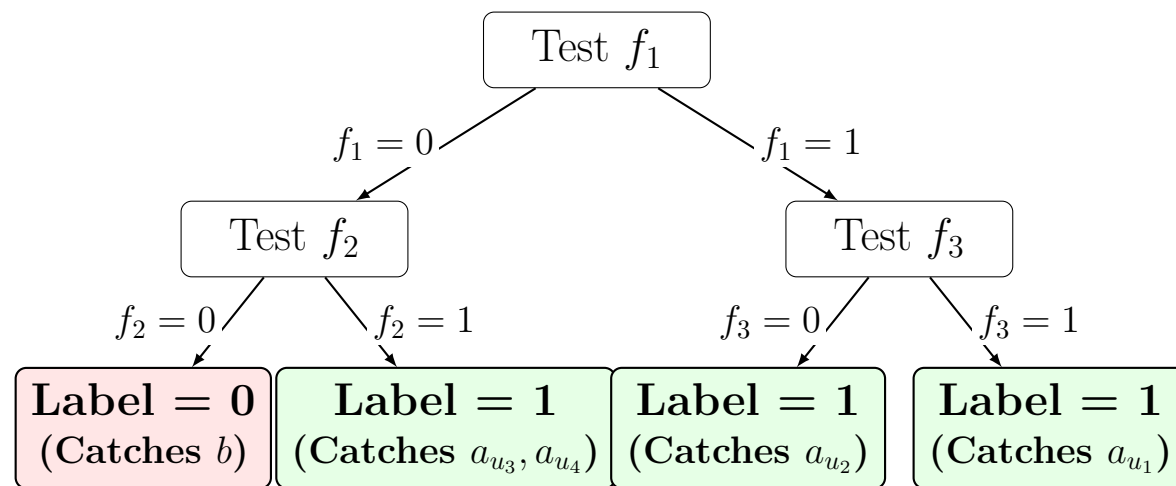


Figure 5.2: Non-comb 4-leaf tree for Example 5.4; tracing b 's leftmost path extracts the cover $\{S_1, S_2\}$.

cation).

- **Recursion.** Send rows with $x_{i,j^*} \leq s^*$ to the left child and the remainder to the right child; recurse on each child.
- **Stopping.** Halt at a node when a stopping rule fires (zero residual variance, minimum node size, maximum depth); declare it a leaf and assign its leaf value by Lemma 5.4.

Choices of Q and stopping rule define specific algorithms.

Example 5.5 (Greedy is not optimal: XOR). Take $n = 4$ rows on $\mathcal{X} = \{0, 1\}^2$ with $y = x_1 \oplus x_2$. At the root, every single-feature split sends one positive and one negative row to each child, leaving the impurity (0/1, Gini, or entropy) unchanged; any greedy rule halting when no split improves Q stops at the root and achieves 50% accuracy. A depth-2 tree (split on x_1 , then on x_2 inside each child) fits the four rows exactly, so the optimal tree exists in $\mathcal{F}_{\text{tree}}$. Greedy misses it because the first split's payoff is invisible without committing to the second.

Why greedy is still standard

The XOR pathology is real but rarely dominates real-data error: most problems admit splits whose immediate payoff correlates with their long-run payoff. The success of CART, Random Forest, and gradient boosting on tabular benchmarks rests on that empirical regularity, not on a worst-case guarantee. Two CART-era patches mitigate the worst cases: **look-ahead** evaluates two-level splits per node (expensive, rare in practice), and **surrogate splits** record alternatives whose decision agrees with the chosen split (routine).

2 CART

CART (Breiman, Friedman, Olshen, and Stone, 1984) instantiates Definition 5.6 with specific objectives for regression and classification, plus a pruning step.

2.1 Regression trees and SSE splits

For regression ($\mathcal{Y} = \mathbb{R}$, squared loss $\ell(y, c) = (y - c)^2$), Example 5.2 fixes the leaf value at the leaf mean. The remaining choice is which split (j^*, s^*) to pick at each node. CART picks the split with the largest reduction in within-node sum of squares.

Definition 5.7 (Within-node SSE). For an index set $I \subseteq \{1, \dots, n\}$, write $n_I := |I|$, $\bar{y}_I := n_I^{-1} \sum_{i \in I} y_i$, and

$$\text{SSE}(I) := \sum_{i \in I} (y_i - \bar{y}_I)^2.$$

When I is a leaf, Example 5.2 sets the prediction to $\bar{y}_I$, making $\text{SSE}(I)$ that leaf's contribution to the empirical risk.

Definition 5.8 (SSE split gain). For a candidate split (j, s) at a node holding rows I , let $I_L := \{i \in I : x_{i,j} \leq s\}$ and $I_R := I \setminus I_L$, with sizes n_L, n_R and means $\bar{y}_L, \bar{y}_R$. The **SSE split gain** is

$$\Delta(I; j, s) := \text{SSE}(I) - \text{SSE}(I_L) - \text{SSE}(I_R).$$

CART picks $(j^*, s^*) \in \arg \max_{(j,s)} \Delta(I; j, s)$, equivalently $\arg \min_{(j,s)} [\text{SSE}(I_L) + \text{SSE}(I_R)]$ since $\text{SSE}(I)$ is constant in the split.

Lemma 5.9 (Closed form for the SSE split gain). With the notation of Definition 5.8,

$$\Delta(I; j, s) = \frac{n_L n_R}{n_I} (\bar{y}_L - \bar{y}_R)^2.$$

The gain is non-negative, with equality iff $\bar{y}_L = \bar{y}_R$.

Proof. Apply the identity $\sum_{i \in J} (y_i - \bar{y}_J)^2 = \sum_{i \in J} y_i^2 - n_J \bar{y}_J^2$ at $J = I, I_L, I_R$. The $\sum y_i^2$ terms cancel since $\{I_L, I_R\}$ partitions I :

$$\Delta(I; j, s) = n_L \bar{y}_L^2 + n_R \bar{y}_R^2 - n_I \bar{y}_I^2.$$

Substituting $n_I \bar{y}_I = n_L \bar{y}_L + n_R \bar{y}_R$ gives $n_I \bar{y}_I^2 = (n_L \bar{y}_L + n_R \bar{y}_R)^2 / n_I$. Multiplying Δ by n_I and using $n_I = n_L + n_R$,

$$n_I \Delta(I; j, s) = (n_L + n_R)(n_L \bar{y}_L^2 + n_R \bar{y}_R^2) - (n_L \bar{y}_L + n_R \bar{y}_R)^2 = n_L n_R (\bar{y}_L - \bar{y}_R)^2,$$

where the second equality is by direct expansion. Dividing by n_I gives the formula; non-negativity and the equality condition follow from $(\bar{y}_L - \bar{y}_R)^2 \geq 0$. $\square$

Per-feature scan in $O(n \log n)$

Sort I by $x_{:,j}$ in $O(n_I \log n_I)$, and let $v_1 < \dots < v_u$ be the $u \leq n_I$ distinct sorted values; the $u - 1$ midpoints $s_k = (v_k + v_{k+1})/2$ are the candidate thresholds (a tie between r rows at value v_k contributes one threshold and r row-updates, not r thresholds). Pre-compute $T_I := \sum_{i \in I} y_i$ in $O(n_I)$. Sweep the candidates left-to-right, maintaining $(n_L, T_L := \sum_{i \in I_L} y_i)$ via $O(1)$ updates per row entering I_L . From (n_L, T_L) , Lemma 5.9 gives Δ in $O(1)$ via $n_R = n_I - n_L$, $\bar{y}_L = T_L/n_L$, $\bar{y}_R = (T_I - T_L)/n_R$. The sweep does n_I updates and $u - 1$ gain evaluations; the sort dominates, so the per-feature cost is $O(n_I \log n_I)$. Across p features, the split search at a node costs $O(p \cdot n_I \log n_I)$.

2.2 Impurity for classification: Gini and entropy

For K -class classification ($\mathcal{Y} = \{1, \dots, K\}$, 0/1 loss), Example 5.2 sets the leaf prediction to the modal class. The per-node split criterion measures the impurity of the class distribution at the node; CART picks the split that decreases this impurity most.

Definition 5.10 (Impurity function). An **impurity function** on the probability simplex $\Delta^{K-1} = \{p \in \mathbb{R}_{\geq 0}^K : \sum_k p_k = 1\}$ is a continuous concave function $\phi : \Delta^{K-1} \rightarrow \mathbb{R}_{\geq 0}$ with $\phi(e_k) = 0$ at every vertex e_k (the unit vector with 1 in coordinate k).

For a node holding rows I , write $n_{I,k} := |\{i \in I : y_i = k\}|$ for class counts and $p_I := (n_{I,k}/n_I)_{k=1}^K$ for the empirical class proportions.

Definition 5.11 (Three classification impurities). For $p \in \Delta^{K-1}$:

- **Misclassification rate:** $M(p) = 1 - \max_k p_k$.
- **Gini index:** $G(p) = 1 - \sum_k p_k^2 = \sum_k p_k(1 - p_k)$.
- **Entropy:** $H(p) = -\sum_k p_k \log p_k$, with $0 \log 0 := 0$.

Lemma 5.12 (Concavity). M, G, H are concave on Δ^{K-1} . Both G and H are strictly concave on the simplex interior; M is concave but not strictly so.

Proof. For G : $-\sum_k p_k^2$ has Hessian $-2I_K$, strictly concave on $\mathbb{R}^K$ and so on Δ^{K-1} ; adding the constant 1 preserves strict concavity.

For H : $-x \log x$ has second derivative $-1/x < 0$ on $(0, 1]$, so the Hessian of $-\sum_k p_k \log p_k$ is diagonal with entries $-1/p_k < 0$ on the open simplex; H is strictly concave there, with concavity extending to the boundary by continuity.

For M : $\max_k p_k$ is a maximum of linear functions, hence convex, so $M = 1 - \max_k p_k$ is concave. On any region of Δ^{K-1} where one class k^* is the strict modal class, $M(p) = 1 - p_{k^*}$ is linear in p , so M fails strict concavity on any segment inside such a region. $\square$

Definition 5.13 (Impurity split gain). With I_L, I_R as in Definition 5.8, the **impurity split gain** under impurity ϕ is

$$\Delta_\phi(I; j, s) := \phi(p_I) - \frac{n_L}{n_I} \phi(p_L) - \frac{n_R}{n_I} \phi(p_R).$$

The CART classification splitter at node I picks $(j^*, s^*) \in \arg \max_{(j,s)} \Delta_\phi(I; j, s)$.

Lemma 5.14 (Non-negative split gain). For any concave impurity ϕ , $\Delta_\phi(I; j, s) \geq 0$. If ϕ is strictly concave and both children are non-empty, $\Delta_\phi = 0$ iff $p_L = p_R$.

Proof. The parent class proportions are the size-weighted average of the child proportions: $p_I = (n_L/n_I)p_L + (n_R/n_I)p_R$. Applying Jensen to ϕ with weights n_L/n_I and n_R/n_I gives $\phi(p_I) \geq (n_L/n_I)\phi(p_L) + (n_R/n_I)\phi(p_R)$, i.e., $\Delta_\phi \geq 0$. Strict concavity makes the inequality strict unless $p_L = p_R$. $\square$

The non-strict concavity of M explains why it ties certain splits.

Example 5.6 (Misclassification ties; Gini and entropy break the tie). Take a parent node with class counts (400, 400) ($K = 2$). Two candidate splits:

- Split A: children (300, 100) and (100, 300).
- Split B: children (200, 400) and (200, 0).

Parent and weighted-child impurities under each criterion:

	$\phi(p_I)$	Split A (weighted)	Split B (weighted)
M	0.500	0.250	0.250
G	0.500	0.375	0.333
H	0.693	0.562	0.477

M gives identical reductions for A and B (0.25 each); G and H both prefer B (G : 0.167 vs. 0.125; H : 0.216 vs. 0.131). The (200, 0) leaf in B is pure, which is visible to G and H but invisible to M .

Practical defaults

rpart defaults to Gini for classification; so does **ranger**. Entropy and Gini rarely disagree in practice; the choice between them is mostly historical or implementation-driven.

2.3 Cost-complexity pruning

Greedy growing (Definition 5.6) with a permissive stopping rule produces an over-grown tree $T_{\max}$ that overfits the training data. Pruning collapses subtrees of $T_{\max}$ into leaves; CART's cost-complexity pruning offers a one-parameter family indexed by $\alpha \geq 0$.

Definition 5.15 (Pruned subtree). A **pruned subtree** $T \preceq T_{\max}$ is obtained from $T_{\max}$ by selecting a (possibly empty) set of internal nodes and collapsing the subtree rooted at each into a single leaf. The set of pruned subtrees of $T_{\max}$ is finite.

Definition 5.16 (Risk and cost-complexity). For a tree T with leaves $\{I_m\}_{m=1}^{|T|}$, let $R(T)$ denote its empirical risk under the per-leaf optimal value rule (Lemma 5.4). For regression, $R(T) = \frac{1}{n} \sum_m \text{SSE}(I_m)$. For classification, $R(T) = \frac{1}{n} \sum_m n_m \cdot M(p_m)$ with M from Definition 5.11. The **cost-complexity** of T at $\alpha \geq 0$ is

$$R_\alpha(T) := R(T) + \alpha |T|.$$

The **optimal pruned subtree** at α is

$$T_\alpha := \arg \min_{T \preceq T_{\max}} R_\alpha(T),$$

unique by tie-breaking in favor of the smallest such T .

The map $\alpha \mapsto T_\alpha$ is piecewise constant with finitely many breakpoints, despite the exponential number of pruned subtrees.

Theorem 5.17 (Nested optimal pruning sequence). There exist a finite sequence $0 = \alpha_0 < \alpha_1 < \cdots < \alpha_K$ and a strictly nested sequence $T_{\max} = T_0 \succ T_1 \succ \cdots \succ T_K = \{\text{root}\}$ such that

$$T_\alpha = T_k \quad \text{for } \alpha \in [\alpha_k, \alpha_{k+1}),$$

with T_K optimal for $\alpha \geq \alpha_K$.

These are produced by **weakest-link pruning**: at each step, collapse the subtree with the smallest per-leaf cost reduction.

Definition 5.18 (Weakest-link criterion). For an internal node t in a tree T , let T_t be the subtree of T rooted at t and $\{t\}$ the result of collapsing T_t into a single leaf. The **weakest-link criterion** at t is

$$g(t; T) := \frac{R(\{t\}) - R(T_t)}{|T_t| - 1}.$$

Lemma 5.19 (Weakest link gives the next breakpoint). For a tree T with $|T| > 1$ and any

internal node t , let T'_t denote T with T_t collapsed to $\{t\}$. Then

$$R_\alpha(T'_t) \leq R_\alpha(T) \iff \alpha \geq g(t; T),$$

with equality at $\alpha = g(t; T)$. The smallest $\alpha \geq 0$ at which any single prune of T achieves cost-complexity no greater than $R_\alpha(T)$ is $\alpha^*(T) := \min_t g(t; T)$.

Proof. Collapsing T_t to $\{t\}$ gives $R(T'_t) - R(T) = R(\{t\}) - R(T_t)$ and $|T'_t| - |T| = -(|T_t| - 1)$, so the cost-complexity change is

$$R_\alpha(T'_t) - R_\alpha(T) = (R(\{t\}) - R(T_t)) - \alpha(|T_t| - 1).$$

By Lemma 5.4, refining a partition can only lower the sum of per-cell minimal losses, so $R(\{t\}) \geq R(T_t)$ and $|T_t| - 1 > 0$. Hence $R_\alpha(T'_t) \leq R_\alpha(T)$ iff $\alpha(|T_t| - 1) \geq R(\{t\}) - R(T_t)$, iff $\alpha \geq g(t; T)$, with equality at $\alpha = g(t; T)$. The conclusion about $\alpha^*(T)$ is immediate. $\square$

Definition 5.20 (Weakest-link pruning algorithm). Starting at $T_0 = T_{\max}$ and $\alpha_0 = 0$, iterate for $k = 0, 1, 2, \dots$:

- Compute $\alpha_{k+1} = \alpha^*(T_k)$ (Lemma 5.19) and pick $t_k^* \in \arg \min_t g(t; T_k)$.
- Set T_{k+1} to be T_k with $T_{t_k^*}$ collapsed to $\{t_k^*\}$.
- Halt when $|T_{k+1}| = 1$.

The output is the nested sequence and breakpoints of Theorem 5.17.

Proof of Theorem 5.17. The proof has four parts: modularity of R_α on the pruning lattice, existence of a smallest optimal subtree at each α , monotonicity of T_α in α , and an induction matching the weakest-link sequence with the optima.

Modularity. Identify each pruned subtree $T \preccurlyeq T_{\max}$ with its set of internal nodes $I(T) \subseteq I(T_{\max})$, which is closed under taking ancestors. The pruning lattice has meet $T_a \wedge T_b$ corresponding to $I(T_a) \cap I(T_b)$ (more pruned) and join $T_a \vee T_b$ corresponding to $I(T_a) \cup I(T_b)$ (less pruned). Let L_v denote the leaf risk of $v \in T_{\max}$ if v were a leaf, so $R(T) = \frac{1}{n} \sum_{v \text{ leaf in } T} L_v$. Checking the leaf status of each v across the four trees case by case (whether v is internal, leaf, or absent in each of

T_a, T_b) verifies

$$\mathbb{1}\{v \text{ leaf in } T_a\} + \mathbb{1}\{v \text{ leaf in } T_b\} = \mathbb{1}\{v \text{ leaf in } T_a \vee T_b\} + \mathbb{1}\{v \text{ leaf in } T_a \wedge T_b\}.$$

Summing L_v -weighted contributions: $R(T_a) + R(T_b) = R(T_a \vee T_b) + R(T_a \wedge T_b)$. The identity $|T| = |I(T)| + 1$ for binary trees gives the same modular relation for $|\cdot|$, so R_α is modular: $R_\alpha(T_a) + R_\alpha(T_b) = R_\alpha(T_a \vee T_b) + R_\alpha(T_a \wedge T_b)$.

Smallest optimal subtree. For each α , let $\mathcal{O}_\alpha := \{T : R_\alpha(T) = \min\}$. For $T_a, T_b \in \mathcal{O}_\alpha$, modularity gives $R_\alpha(T_a \vee T_b) + R_\alpha(T_a \wedge T_b) = 2 \min R_\alpha$; neither term can fall below the minimum, so both equal it. Thus $\mathcal{O}_\alpha$ is closed under meet and join: a sublattice of the finite pruning lattice.

The meet of all its elements is the unique smallest optimal subtree under $\preceq$; define T_α as this meet.

Monotonicity. Suppose $\alpha_1 < \alpha_2$ and (for contradiction) $T_{\alpha_2} \not\preceq T_{\alpha_1}$. Set $T^* := T_{\alpha_1} \wedge T_{\alpha_2}$ and $T^+ := T_{\alpha_1} \vee T_{\alpha_2}$. By modularity at any α ,

$$R_\alpha(T^*) - R_\alpha(T_{\alpha_2}) = R_\alpha(T_{\alpha_1}) - R_\alpha(T^+) =: \Delta_\alpha.$$

Optimality of T_{α_1} at α_1 (with T^+ as candidate) gives $\Delta_{\alpha_1} \leq 0$; optimality of T_{α_2} at α_2 (with T^* as candidate) gives $\Delta_{\alpha_2} \geq 0$. The function $\alpha \mapsto \Delta_\alpha$ is linear with slope $|T^*| - |T_{\alpha_2}| \leq 0$. If the slope

is strictly negative, $\Delta_{\alpha_2} < \Delta_{\alpha_1} \leq 0$, contradicting $\Delta_{\alpha_2} \geq 0$. So the slope is zero: $|T^*| = |T_{\alpha_2}|$. Combined with $T^* \preceq T_{\alpha_2}$, this forces $T^* = T_{\alpha_2}$, i.e., $T_{\alpha_2} \preceq T_{\alpha_1}$, contradicting the assumption.

Multi-step pruning gain. Any $T' \prec T_k$ corresponds to collapsing a unique antichain $\{t_1, \dots, t_m\}$ of internal nodes in T_k (the topmost nodes that become leaves in T'). Their subtrees $T_{(k),t_j}$ are pairwise disjoint, so the cost-complexity changes are additive:

$$R_\alpha(T') - R_\alpha(T_k) = \sum_{j=1}^m (g(t_j; T_k) - \alpha) (|T_{(k),t_j}| - 1).$$

For $\alpha < \alpha_{k+1} = \min_{t \text{ internal in } T_k} g(t; T_k)$, every $g(t_j; T_k) > \alpha$, so each term is strictly positive: $R_\alpha(T') > R_\alpha(T_k)$.

Induction. At $\alpha = 0$: $T_0 = T_{\max}$ minimizes R (refining the partition only lowers per-cell minimal loss; the $\alpha = 0$ minimum is achieved at the finest partition); the smallest optimal subtree at $\alpha = 0$ is $T_{\max}$. Inductively, assume $T_\alpha = T_k$ on $[\alpha_k, \alpha_{k+1})$. By monotonicity, $T_\alpha \preceq T_k$ for $\alpha \geq \alpha_k$. By the multi-step gain, no $T' \prec T_k$ achieves $R_\alpha(T') \leq R_\alpha(T_k)$ for $\alpha < \alpha_{k+1}$, so $T_\alpha = T_k$ throughout $[\alpha_k, \alpha_{k+1})$. At $\alpha = \alpha_{k+1}$, T_{k+1} ties with T_k (Lemma 5.19); tie-breaking in favor of fewer leaves

gives $T_{\alpha_{k+1}} = T_{k+1}$. Each weakest-link step strictly reduces leaf count, so the sequence terminates at $T_K = \{\text{root}\}$. $\square$

The complexity parameter α is chosen by cross-validation: for each fold, generate the pruning sequence on the training portion and evaluate each subtree on the held-out fold. Average held-out risk across folds at corresponding α values (per-fold sequences have different breakpoints; standard practice aligns them via geometric means of consecutive thresholds), then pick α^* minimizing the average. The 1-SE rule (smallest $|T|$ within one standard error of the minimum) is preferred when parsimony matters.

Where pruning fits

CART grows with Gini or entropy (whose strict concavity breaks ties; §5.2.2) and prunes with misclassification rate, which optimizes the classification cost directly. **rpart** follows this convention; its **cp** parameter is α divided by the root's risk. Ensemble methods (Random Forest, gradient boosting) replace single-tree pruning with depth caps plus averaging or shrinkage, treating each base tree as deliberately under-regularized; cost-complexity remains the standard regularizer for single CART trees.

3 Bagging and Random Forests

A single CART tree achieves low bias when grown deep but high variance: small perturbations of the training sample produce different trees. Bagging (Breiman, 1996) and Random Forest (Breiman, 2001) reduce that variance by averaging many trees fit on resampled data, with Random Forest adding feature subsampling to lower the correlation between trees.

3.1 Bootstrap and out-of-bag estimation

Bagging fits an ensemble of trees on bootstrap resamples of the training set. The same construction yields a free held-out estimate of generalization error: for each row, predict using only the trees whose bootstrap sample excluded that row.

Definition 5.21 (Bootstrap sample). Given a training set of n rows, a **bootstrap sample** $\mathcal{D}^* = (i_1^*, \dots, i_n^*)$ is drawn by sampling n row indices independently and uniformly with replacement from $\{1, \dots, n\}$.

A training row may appear multiple times in a bootstrap sample, or not at all. Rows excluded from $\mathcal{D}^*$ are the **out-of-bag** (OOB) rows for that sample.

Lemma 5.22 (Out-of-bag inclusion probability). The probability that a fixed row i is OOB for a single bootstrap sample is

$$\mathbf{P}(i \notin \mathcal{D}^*) = \left(1 - \frac{1}{n}\right)^n \xrightarrow{n \rightarrow \infty} e^{-1} \approx 0.368.$$

For B independent bootstrap samples, the expected number of samples for which row i is OOB is $B \cdot (1 - 1/n)^n$.

Proof. Each of the n draws excludes row i with probability $1 - 1/n$; mutual independence of the draws gives $\mathbf{P}(i \notin \mathcal{D}^*) = (1 - 1/n)^n$. The limit $\lim_n (1 - 1/n)^n = e^{-1}$ is elementary calculus. For B samples, the expected count is $B(1 - 1/n)^n$ by linearity of expectation. $\square$

For an ensemble of B trees $\hat{f}_1, \dots, \hat{f}_B$ trained on bootstrap samples $\mathcal{D}_1^*, \dots, \mathcal{D}_B^*$, define $\mathcal{B}_i := \{b : i \notin \mathcal{D}_b^*\}$ for each row i (the trees that did not see row i).

Definition 5.23 (OOB prediction and OOB error). The **OOB prediction** for row i is

$$\hat{y}_i^{\text{ooB}} := \frac{1}{|\mathcal{B}_i|} \sum_{b \in \mathcal{B}_i} \hat{f}_b(x_i)$$

(or the modal class for classification ensembles). The **OOB error** of the ensemble is

$$\hat{\mathcal{R}}_{\text{ooB}} := \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{y}_i^{\text{ooB}}).$$

OOB and CV

$\hat{y}_i^{\text{ooB}}$ is the average of roughly $0.368B$ trees that did not see row i , a cross-validated estimate at no extra training cost. With $B = 500$, each row's prediction averages ≈ 184 trees; OOB error converges to the bagged ensemble's test error at the rate of a similarly-sized held-out test set. The estimate is unreliable under distribution shift between training and deployment.

3.2 Variance reduction by averaging

Bagging's value comes from averaging: identically distributed predictors have variance that decreases with the number of replicates, but the decrease has a floor set by their pairwise correlation.

Lemma 5.24 (Variance of an average of correlated predictors). Let $\hat{f}_1, \dots, \hat{f}_B$ be identically distributed random variables with $\text{var}(\hat{f}_b) = \sigma^2$ and pairwise correlation $\text{cor}(\hat{f}_b, \hat{f}_{b'}) = \rho$ for all $b \neq b'$. Then

$$\text{var}\left(\frac{1}{B} \sum_{b=1}^B \hat{f}_b\right) = \rho\sigma^2 + \frac{(1-\rho)\sigma^2}{B}.$$

Proof. By bilinearity of covariance,

$$\text{var}\left(\frac{1}{B} \sum_b \hat{f}_b\right) = \frac{1}{B^2} \sum_{b,b'} \text{cov}(\hat{f}_b, \hat{f}_{b'}) = \frac{1}{B^2} (B\sigma^2 + B(B-1)\rho\sigma^2),$$

where the diagonal contributes B terms of σ^2 and the off-diagonal contributes $B(B-1)$ terms of $\rho\sigma^2$. Simplifying yields $\rho\sigma^2 + (1-\rho)\sigma^2/B$. $\square$

Two consequences:

- As $B \rightarrow \infty$, the variance approaches the floor $\rho\sigma^2$. Increasing B alone cannot shrink variance below this floor.
- For uncorrelated predictors ($\rho = 0$), variance shrinks as σ^2/B ; for fully correlated predictors ($\rho = 1$), averaging gives no reduction.

Bagged trees fit on bootstraps of the same data are correlated through the shared data, so ρ is bounded well above zero; B alone runs into the variance floor. Lowering ρ is the next move.

3.3 Decorrelation by feature subsampling

Bagged CART trees on bootstraps of the same data tend to make similar early splits: the strongest feature dominates the split criterion on most resamples, so different trees agree on root-level structure and their predictions correlate. Random Forest reduces this correlation by restricting the split search at each node to a random subset of features.

Definition 5.25 (Random Forest). A **Random Forest** is an ensemble of B trees, each fit on an independent bootstrap sample of the training set. At each internal node of each tree, the split search of Definition 5.6 is restricted to a random subset of m features ($1 \leq m \leq p$) drawn

uniformly without replacement from $\{1, \dots, p\}$, with a fresh draw at every node. Trees are grown to maximum depth without pruning; ensemble averaging supplies the regularization.

The hyperparameter m (often called **mtry**) controls the bias–variance trade-off in the ensemble:

- Smaller m forces trees to use different features, lowering pairwise correlation ρ and thus the variance floor of Lemma 5.24.
- Smaller m also raises each individual tree’s variance: a tree forced to split on suboptimal features grows differently from the optimal CART tree, with higher single-tree error.

The defaults $m = \lceil \sqrt{p} \rceil$ for classification and $m = \lfloor p/3 \rfloor$ for regression (Breiman, 2001) work well across many datasets; cross-validation over $m \in \{2, 4, 8, \dots, p\}$ is the standard tuning approach.

Implementation cost

A naive implementation re-sorts each candidate feature at every node ($O(n_I \log n_I)$ per feature per node). Production libraries (**ranger**, **xgboost**) pre-sort each feature once at the start of training and reuse the sorted indices at every split, reducing the per-feature per-node cost to $O(n_I)$.

3.4 Variable importance: MDI and permutation

Tree ensembles do not produce coefficients, but two scalar summaries quantify each feature's contribution to the ensemble's predictions.

Definition 5.26 (Mean decrease in impurity). For an ensemble of B trees $\{T_b\}$ and feature $j \in \{1, \dots, p\}$, the **MDI** of j is

$$\text{MDI}(j) := \frac{1}{B} \sum_{b=1}^B \sum_{\text{internal } t \in T_b : j_t = j} \frac{n_{I_t}}{n} \cdot \Delta(I_t; j_t, s_t),$$

where the inner sum runs over internal nodes of T_b that split on feature j , Δ is the relevant split gain (Definition 5.8 for regression, Definition 5.13 for classification), and n_{I_t} is the count of rows reaching node t .

Definition 5.27 (Permutation importance). For each feature j , let $\widehat{\mathcal{R}}_{\text{oob}}^{(\pi j)}$ denote the OOB error of the ensemble evaluated after permuting column j uniformly at random across the training rows.

The **permutation importance** of j is

$$\text{PI}(j) := \widehat{\mathcal{R}}_{\text{oob}}^{(\pi^j)} - \widehat{\mathcal{R}}_{\text{oob}}.$$

The two measures answer different questions and have different failure modes:

- MDI rewards features with many possible split points: a continuous high-cardinality feature has more candidate thresholds than a low-cardinality categorical one, inflating its impurity-reduction sum even at random.
- Correlated features dilute each other's MDI and PI: each tree picks one of several near-equivalent features at each node, splitting the credit.
- Permutation importance is reliable only under feature independence; when feature j is correlated with relevant features, permuting j produces row vectors outside the training distribution and PI loses interpretability.

3.5 SHAP and TreeSHAP

MDI and permutation importance summarize feature use globally; modern practice often wants per-row attributions: how does each feature contribute to the model's prediction for a specific row x ?

SHAP (Lundberg and Lee, 2017) imports cooperative game theory to answer this, and TreeSHAP (Lundberg et al., 2020) makes the computation polynomial-time on tree ensembles.

Cooperative games and the Shapley value. A cooperative game on a finite set N of **players** assigns a real number to each coalition (subset of players); the question is how to fairly divide the total value $v(N)$ among the players given each player's contributions across coalitions.

Definition 5.28 (Cooperative game). A **cooperative game** on a finite player set N is a function $v : 2^N \rightarrow \mathbb{R}$ with $v(\emptyset) = 0$. The number $v(S)$ is the **value** of coalition $S \subseteq N$.

Definition 5.29 (Value function and Shapley axioms). A **value function** assigns to each cooperative game v on N a vector $\phi(v) = (\phi_i(v))_{i \in N} \in \mathbb{R}^N$ of **individual contributions**. The Shapley axioms require, for all games v, w on N :

- **Efficiency**: $\sum_{i \in N} \phi_i(v) = v(N)$.
- **Symmetry**: if $v(S \cup \{i\}) = v(S \cup \{j\})$ for every $S \subseteq N \setminus \{i, j\}$, then $\phi_i(v) = \phi_j(v)$.
- **Dummy player**: if $v(S \cup \{i\}) = v(S)$ for every $S \subseteq N \setminus \{i\}$, then $\phi_i(v) = 0$.

- **Linearity:** $\phi(\alpha v + \beta w) = \alpha\phi(v) + \beta\phi(w)$ for all scalars $\alpha, \beta \in \mathbb{R}$, where $(\alpha v + \beta w)(S) := \alpha v(S) + \beta w(S)$.

Theorem 5.30 (Shapley, 1953). There is a unique value function satisfying all four Shapley axioms; it is given by the **Shapley value**

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)].$$

Equivalently, summing over permutations π of N ,

$$\phi_i(v) = \frac{1}{|N|!} \sum_{\pi \in \Pi_N} [v(P_\pi^i \cup \{i\}) - v(P_\pi^i)],$$

where P_π^i is the set of players preceding i in π .

Proof. Equivalence of the two formulas. Group the $|N|!$ permutations by $S := P_\pi^i$. For each $S \subseteq N \setminus \{i\}$ of size k , exactly $k! (|N| - k - 1)!$ permutations have $P_\pi^i = S$ ($k!$ orderings of S before i , $(|N| - k - 1)!$ orderings of $N \setminus (S \cup \{i\})$ after). The marginal contribution depends only on S , so

$$\frac{1}{|N|!} \sum_{\pi} [v(P_\pi^i \cup \{i\}) - v(P_\pi^i)] = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! (|N| - |S| - 1)!}{|N|!} [v(S \cup \{i\}) - v(S)].$$

Existence. The closed-form formula satisfies all four axioms.

Efficiency. In the permutation form, fix π and sum the marginal contributions in the order players appear in π : this telescopes from $v(\emptyset) = 0$ through $v(N)$, giving $\sum_i [v(P_\pi^i \cup \{i\}) - v(P_\pi^i)] = v(N)$. Averaging over π gives $\sum_i \phi_i(v) = v(N)$.

Symmetry. Suppose i, j are interchangeable in v . Define a bijection σ on subsets of $N \setminus \{i\}$: $\sigma(S) = S$ if $j \notin S$, and $\sigma(R \cup \{j\}) = R \cup \{i\}$ for $R \subseteq N \setminus \{i, j\}$. The bijection preserves cardinality, so the Shapley weights $|S|! (|N| - |S| - 1)! / |N|!$ in $\phi_i(v)$ and $\phi_j(v)$ match summand-for-summand. The marginal contributions also match: for $S \subseteq N \setminus \{i, j\}$, $v(S \cup \{i\}) - v(S) = v(S \cup \{j\}) - v(S)$ by interchangeability; for $S = R \cup \{j\}$, $v(R \cup \{i, j\}) - v(R \cup \{j\}) = v(R \cup \{i, j\}) - v(R \cup \{i\})$

since interchangeability with $S = R$ gives $v(R \cup \{j\}) = v(R \cup \{i\})$. Hence $\phi_i(v) = \phi_j(v)$.

Dummy. If $v(S \cup \{i\}) = v(S)$ for every $S \not\ni i$, every summand in the subset form vanishes; so $\phi_i(v) = 0$.

Linearity. The formula is linear in v summand-by-summand.

Uniqueness. Let ψ be any value function satisfying the four axioms.

Step 1 (basis). For each non-empty $T \subseteq N$, define $u_T(S) := \mathbb{1}\{T \subseteq S\}$. The space of cooperative games on N (functions $v : 2^N \rightarrow \mathbb{R}$ with $v(\emptyset) = 0$) is a real vector space of dimension $2^{|N|} - 1$, matching the count of unanimity games. To show linear independence, suppose $\sum_T c_T u_T = 0$ but some $c_T \neq 0$, and let T_0 be a minimal (under inclusion) non-empty subset with $c_{T_0} \neq 0$. Then

$$0 = \sum_T c_T u_T(T_0) = \sum_{T \subseteq T_0} c_T = c_{T_0},$$

the second equality because $u_T(T_0) = 1$ iff $T \subseteq T_0$, and the third because $c_T = 0$ for every $T \subsetneq T_0$ by minimality of T_0 . This contradicts $c_{T_0} \neq 0$, so the unanimity games are independent and form a basis.

Step 2 (ψ on $c u_T$). Fix non-empty T and scalar c .

- For $i \notin T$: $u_T(S \cup \{i\}) = \mathbf{1}\{T \subseteq S \cup \{i\}\} = \mathbf{1}\{T \subseteq S\} = u_T(S)$ since $i \notin T$, so i is a dummy in $c u_T$; the dummy axiom gives $\psi_i(c u_T) = 0$.
- For $i \in T$: any two members of T are interchangeable in $c u_T$ (the value depends only on whether $T \subseteq S$); symmetry forces $\psi_i(c u_T) = \alpha$ for a common α across all $i \in T$. Efficiency gives $|T| \cdot \alpha = (c u_T)(N) = c$, so $\alpha = c/|T|$.

Step 3 (extension). Every game v has a unique decomposition $v = \sum_T c_T u_T$ by Step 1. Linearity gives

$$\psi(v) = \sum_T c_T \psi(u_T),$$

and Step 2 determines $\psi(u_T)$. The same argument applied to ϕ (which satisfies the four axioms by Existence) yields the same values, so $\psi = \phi$. $\square$

SHAP: Shapley values for feature attribution. To attribute a model prediction $f(x)$ at a row $x \in \mathbb{R}^p$ to its features, take $N = \{1, \dots, p\}$ (the features) as the player set and choose a coalitional value function v_x derived from f . The two standard choices are:

- **Conditional** (observational): $v_x(S) = \mathbf{E}[f(X) \mid X_S = x_S]$, the model's expected output given

$$X_S = x_S.$$

- **Interventional** (causal-flavored): $v_x(S) = \mathbf{E}_{X_{N \setminus S}}[f(x_S, X_{N \setminus S})]$, the model's expected output with S -features fixed at x_S and the rest drawn from the marginal.

The two coincide when features are independent; under correlation they differ. The interventional form fits causal-style attribution; the conditional form fits descriptive analysis of how the model uses features on observed data.

Definition 5.31 (SHAP value). Given a value function v_x derived from f at row x , the **SHAP value** of feature i is the Shapley value $\phi_i(v_x)$ of Theorem 5.30.

The efficiency axiom gives a per-row decomposition that MDI and PI lack:

$$f(x) = \mathbf{E}[f(X)] + \sum_{i=1}^p \phi_i(v_x).$$

The model's prediction at x equals the dataset baseline plus contributions from each feature, additively; the decomposition is unique under the four axioms.

Example 5.7 (SHAP for a linear model). For $f(x) = \beta_0 + \sum_{j=1}^p \beta_j x_j$ with $\mathbf{E}[X_j] = \mu_j$, the interventional value function with feature marginals reduces to

$$v_x(S) = \beta_0 + \sum_{j \in S} \beta_j x_j + \sum_{j \notin S} \beta_j \mu_j.$$

Each marginal contribution $v_x(S \cup \{i\}) - v_x(S) = \beta_i(x_i - \mu_i)$ does not depend on S , so the Shapley sum collapses to

$$\phi_i(v_x) = \beta_i(x_i - \mu_i).$$

Efficiency reads $f(x) - \mathbf{E}[f(X)] = \sum_i \beta_i(x_i - \mu_i)$, recovering the linear identity directly.

TreeSHAP. Computing $\phi_i(v_x)$ from Theorem 5.30 naively requires iterating over all 2^{p-1} subsets $S \subseteq N \setminus \{i\}$, infeasible already at $p = 30$. For tree ensembles a polynomial-time exact algorithm exists.

Theorem 5.32 (TreeSHAP, Lundberg et al., 2020). For a single decision tree with L leaves and depth D , the SHAP values $\{\phi_i(v_x)\}_{i=1}^p$ under the interventional value function with marginals from the training data can be computed exactly in time $O(LD^2)$ and memory $O(D^2)$. For an ensemble of B trees, the cost scales linearly in B .

Proof of Theorem 5.32. Setup. For each leaf ℓ let f_ℓ denote the leaf prediction and $\text{path}(\ell) = (v_1, \dots, v_{D_\ell})$ the sequence of internal nodes from root to ℓ . For each $v \in \text{path}(\ell)$ with split feature j_v and threshold s_v , let

$$q_v := \mathbb{P}_{X_{j_v}}\{X_{j_v} \text{ on the } \ell\text{-side of } s_v\}, \quad \tilde{a}_v(x) := \mathbb{1}\{x_{j_v} \text{ on the } \ell\text{-side at } v\} \in \{0, 1\},$$

and write $\delta_v(x) := \tilde{a}_v(x) - q_v$. Assume the features along each $\text{path}(\ell)$ are distinct; the algorithm extends to repeated features by combining their constraints, and the bound below is unaffected.

Step 1: Path factorization. Decompose $f(x) = \sum_\ell f_\ell \mathbb{1}\{x \text{ reaches } \ell\}$ and let $\beta_\ell(S; x) := \mathbb{P}_{X_{N \setminus S}}\{(x_S, X_{N \setminus S}) \text{ reaches } \ell\}$. Then

$$v_x(S) = \mathbb{E}_{X_{N \setminus S}}[f(x_S, X_{N \setminus S})] = \sum_\ell f_\ell \cdot \beta_\ell(S; x).$$

The hybrid row reaches ℓ iff at every $v \in \text{path}(\ell)$, the active value of j_v lies on the ℓ -side: x_{j_v} if $j_v \in S$ (deterministic), X_{j_v} if $j_v \notin S$ (random under the marginal). Under the marginal-distribution interventional formulation, $\{X_{j_v} : j_v \notin S\}$ is a collection of mutually independent random variables. With path features distinct, the ℓ -direction events at distinct v are independent, so

$$\beta_\ell(S; x) = \prod_{v \in \text{path}(\ell)} \pi_v(S; x), \quad \pi_v(S; x) := q_v + \mathbf{1}\{j_v \in S\} \delta_v(x).$$

($\pi_v = \tilde{a}_v$ when $j_v \in S$ and $\pi_v = q_v$ otherwise.)

Step 2: Per-leaf SHAP contribution. Define

$$\Phi_{\ell,i}(x) := \sum_{S \subseteq N \setminus \{i\}} w(|S|) [\beta_\ell(S \cup \{i\}; x) - \beta_\ell(S; x)], \quad w(k) := \frac{k! (p - k - 1)!}{p!}.$$

For $j_v \neq i$: $\pi_v(S \cup \{i\}; x) = \pi_v(S; x)$. So if $i \notin \{j_v : v \in \text{path}(\ell)\}$, the difference vanishes and

$\Phi_{\ell,i}(x) = 0$. Otherwise let v_i be the unique path node with $j_{v_i} = i$; only the v_i factor changes:

$$\beta_{\ell}(S \cup \{i\}; x) - \beta_{\ell}(S; x) = [\pi_{v_i}(S \cup \{i\}; x) - \pi_{v_i}(S; x)] \prod_{v \neq v_i} \pi_v(S; x) = \delta_{v_i}(x) \prod_{v \neq v_i} \pi_v(S; x).$$

The factor $\delta_{v_i}(x)$ equals $1 - q_{v_i}$ when x takes the ℓ -direction at v_i and $-q_{v_i}$ otherwise, covering both cases of $\tilde{a}_{v_i}$. Hence

$$\Phi_{\ell,i}(x) = \delta_{v_i}(x) \cdot U_{\ell}^{(i)}(x), \quad U_{\ell}^{(i)}(x) := \sum_{S \subseteq N \setminus \{i\}} w(|S|) \prod_{v \neq v_i} \pi_v(S; x).$$

Step 3: Reducing $U_{\ell}^{(i)}(x)$ to a generating function. Let $J_{\ell}^{(i)} := \{j_v : v \in \text{path}(\ell), v \neq v_i\}$ (size $d := D_{\ell} - 1$, distinct). Expanding the product:

$$\prod_{v \neq v_i} \pi_v(S; x) = \prod_{v \neq v_i} [q_v + \mathbb{1}\{j_v \in S\} \delta_v(x)] = \sum_{T \subseteq J_{\ell}^{(i)}} \mathbb{1}\{T \subseteq S\} \prod_{j \in T} \delta_{v(j)}(x) \prod_{j \in J_{\ell}^{(i)} \setminus T} q_{v(j)},$$

where $v(j)$ is the path node testing j . Substituting and switching the order of summation:

$$U_{\ell}^{(i)}(x) = \sum_{T \subseteq J_{\ell}^{(i)}} \prod_{j \in T} \delta_{v(j)}(x) \prod_{j \in J_{\ell}^{(i)} \setminus T} q_{v(j)} \cdot \sum_{S \supseteq T, S \subseteq N \setminus \{i\}} w(|S|).$$

The inner sum over S counts how many $S \supseteq T$ have $|S| = |T| + k$, namely $\binom{p-|T|-1}{k}$ (we choose k additional elements from $N \setminus T \setminus \{i\}$, regardless of whether they lie in $J_{\ell}^{(i)} \setminus T$ or outside $J_{\ell}^{(i)}$). Therefore the inner sum equals

$$\tilde{W}(|T|) := \sum_{k=0}^{p-|T|-1} \binom{p-|T|-1}{k} w(|T| + k),$$

depending only on $|T|$. Grouping the outer sum by $|T| = t$:

$$U_{\ell}^{(i)}(x) = \sum_{t=0}^d \tilde{W}(t) \cdot c_t^{(i)}, \quad c_t^{(i)} := \sum_{T \subseteq J_{\ell}^{(i)}, |T|=t} \prod_{j \in T} \delta_{v(j)}(x) \prod_{j \in J_{\ell}^{(i)} \setminus T} q_{v(j)}.$$

The coefficients $\{c_t^{(i)}\}_{t=0}^d$ are exactly the coefficients of the polynomial

$$P_\ell^{(i)}(z) := \prod_{j \in J_\ell^{(i)}} (q_{v(j)} + \delta_{v(j)}(x) z) = \sum_{t=0}^d c_t^{(i)} z^t,$$

since the coefficient of z^t in this product is the sum over choices of t factors contributing δz (the rest contributing q).

Step 4: Polynomial-time DP per leaf. Build the master polynomial along the entire path:

$$\bar{P}_\ell(z) := \prod_{v \in \text{path}(\ell)} (q_v + \delta_v(x) z) = \sum_{t=0}^{D_\ell} C_t z^t.$$

Multiply factors one at a time: starting with the constant polynomial 1, multiplying by one new factor $(q + \delta z)$ updates a degree- k polynomial to a degree- $(k + 1)$ polynomial via $C_k^{\text{new}} = q C_k + \delta C_{k-1}$ (with the convention $C_{-1} = C_{k+1} = 0$ on the boundary), taking $O(k + 1)$ scalar operations. The full master polynomial costs $\sum_{k=0}^{D_\ell-1} O(k + 1) = O(D_\ell^2)$ time and $O(D_\ell)$ memory.

For each path feature i (i.e., each path node v_i), recover

$$P_\ell^{(i)}(z) = \bar{P}_\ell(z) / (q_{v_i} + \delta_{v_i}(x) z)$$

by synthetic division: a degree- D_ℓ polynomial divided by a degree-1 polynomial in $O(D_\ell)$ time. (When $\delta_{v_i}(x) = 0$ the divisor is the constant q_{v_i} and the division reduces to scaling; if both are zero, $\delta_{v_i}(x) = 0$ already gives $\Phi_{\ell,i}(x) = 0$, and no division is needed.) Then $U_\ell^{(i)}(x) = \sum_{t=0}^d \tilde{W}(t) c_t^{(i)}$ takes $O(D_\ell)$ time, and $\Phi_{\ell,i}(x) = \delta_{v_i}(x) \cdot U_\ell^{(i)}(x)$. Across the D_ℓ path features at this leaf: $D_\ell \cdot O(D_\ell) = O(D_\ell^2)$ time. Per-leaf total: $O(D_\ell^2)$ time and $O(D_\ell)$ memory.

Step 5: Total cost. The constants $\tilde{W}(t)$ for $t = 0, \dots, D$ depend only on p and D (not on the tree or x); precompute them in $O(D^2)$ time once and reuse across all leaves and trees. Across L leaves the per-tree cost is $O(LD^2)$ time. The recursive implementation of Lundberg et al. (2020) traverses the tree depth-first, sharing the path-state polynomial across siblings; this gives $O(D^2)$ memory at any point. Accumulating $\phi_i(v_x) = \sum_\ell f_\ell \Phi_{\ell,i}(x)$ over leaves takes $O(L)$ additions per feature; only path features have nonzero contribution, and the summation cost is dominated by $O(LD^2)$. For an ensemble of B trees: $O(BLD^2)$ time, $O(D^2)$ memory. $\square$

Implementations are in the **shap** Python package and the **treeshap** R package, both supporting **xgboost**, **lightgbm**, **catboost**, and **ranger**.

SHAP vs. MDI and PI

SHAP gives per-row attributions that sum to the prediction; MDI gives a single global score per feature; PI gives a degraded-baseline global score. Under feature correlation, interventional SHAP, MDI, and PI can disagree substantially; conditional SHAP and PI share an independence assumption and the same blind spots. SHAP has become the default for tree-model interpretation pipelines because TreeSHAP makes exact computation cheap and the per-row decomposition supports both local explanations and global summaries (mean $|\phi_i|$ across rows).

4 Gradient Boosting

Bagging and Random Forest (§5.3) build trees in parallel, each fit on a bootstrap sample. Boosting builds trees sequentially: each new tree corrects the residual error of the current ensemble. Gradient boosting (Friedman, 2001) frames this iteration as gradient descent in function space, generalizing AdaBoost (Freund and Schapire, 1997) to arbitrary differentiable losses.

4.1 Forward stagewise additive modeling

Setup. Training data $\{(x_i, y_i)\}_{i=1}^n$. The loss $\ell(y, F)$ takes a label y and a real-valued raw-score prediction F ; for classification, F is mapped to a probability by a link function (typically logistic). Empirical risk: $\hat{\mathcal{R}}(f) := \frac{1}{n} \sum_i \ell(y_i, f(x_i))$. We minimize $\hat{\mathcal{R}}$ over an additive class

$$\mathcal{F}_M = \left\{ \sum_{m=1}^M \nu_m h_m : h_m \in \mathcal{H}, \nu_m \in \mathbb{R} \right\}$$

of M -term sums of weak learners (typically shallow trees). Joint optimization over all M terms is intractable; the forward stagewise approximation greedily adds one term per round.

Definition 5.33 (Forward stagewise additive modeling). Initialize $\hat{f}^{(0)} := \arg \min_c \frac{1}{n} \sum_i \ell(y_i, c)$ (the constant minimizing empirical risk). For $m = 1, \dots, M$, solve

$$(\hat{\nu}_m, \hat{h}_m) := \arg \min_{\nu \in \mathbb{R}, h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{f}^{(m-1)}(x_i) + \nu h(x_i)),$$

and update $\hat{f}^{(m)} := \hat{f}^{(m-1)} + \hat{\nu}_m \hat{h}_m$.

For squared loss the round- m minimization decouples: $\hat{\nu}_m \hat{h}_m$ is the least-squares regressor of the residuals $y_i - \hat{f}^{(m-1)}(x_i)$ on x_i . For other losses the joint optimization in Definition 5.33 remains intractable. Friedman's gradient boosting circumvents this by approximating the descent direction.

Functional gradient. View the empirical risk as a function of the prediction vector $F \in \mathbb{R}^n$ with $F_i := f(x_i)$:

$$L(F) := \sum_{i=1}^n \ell(y_i, F_i), \quad \hat{\mathcal{R}}(f) = L(F)/n.$$

The gradient $\nabla L \in \mathbb{R}^n$ has i -th component $\partial \ell(y_i, F_i) / \partial F_i$.

Definition 5.34 (Pseudo-residual). At round m , the **pseudo-residual** at training row i is

$$r_i^{(m)} := - \frac{\partial \ell(y_i, F)}{\partial F} \bigg|_{F=\hat{f}^{(m-1)}(x_i)}.$$

The vector $r^{(m)} = (r_i^{(m)})_{i=1}^n$ equals $-\nabla L(\hat{F}^{(m-1)})$, the negative functional gradient of empirical risk at the current ensemble's predictions on the training rows.

Definition 5.35 (Gradient boosting). Initialize $\hat{f}^{(0)}$ as in Definition 5.33. For $m = 1, \dots, M$:

1. Compute pseudo-residuals $r_i^{(m)}$ via Definition 5.34.
2. Fit a weak learner to the residuals: $\hat{h}_m \in \arg \min_{h \in \mathcal{H}} \sum_i (r_i^{(m)} - h(x_i))^2$.
3. Choose a step size $\hat{\nu}_m$ by line search ($\arg \min_{\nu} \sum_i \ell(y_i, \hat{f}^{(m-1)}(x_i) + \nu \hat{h}_m(x_i))$) or by a fixed shrinkage constant.
4. Update $\hat{f}^{(m)} := \hat{f}^{(m-1)} + \hat{\nu}_m \hat{h}_m$.

Lemma 5.36 (Functional gradient and least squares). The directional derivative of $\hat{\mathcal{R}}$ at $\hat{f}^{(m-1)}$ in the direction h equals $-\frac{1}{n} \langle r^{(m)}, h \rangle$, where $\langle r, h \rangle := \sum_i r_i h(x_i)$. Hence maximizing the rate of empirical-risk decrease over $h \in \mathcal{H}$ amounts to maximizing $\langle r^{(m)}, h \rangle$. Among h in any fixed-radius ℓ_2 -ball, this is achieved by the least-squares fit $\hat{h}_m \in \arg \min_{h \in \mathcal{H}} \sum_i (r_i^{(m)} - h(x_i))^2$.

Proof. The directional derivative is

$$\left. \frac{d}{d\nu} \right|_{\nu=0} \hat{\mathcal{R}}(\hat{f}^{(m-1)} + \nu h) = \frac{1}{n} \sum_i \frac{\partial \ell}{\partial F}(y_i, \hat{F}_i^{(m-1)}) \cdot h(x_i) = -\frac{1}{n} \langle r^{(m)}, h \rangle.$$

Expanding the least-squares objective: $\sum_i (r_i - h(x_i))^2 = \|r\|_2^2 - 2\langle r, h \rangle + \|h\|_2^2$. For h in a fixed-radius ℓ_2 -ball, $\|h\|_2$ is held constant, so minimizing this objective is equivalent to maximizing $\langle r, h \rangle$, which by the directional-derivative formula maximizes the rate of empirical-risk decrease. $\square$

4.2 Pseudo-residuals for common losses

The pseudo-residual depends on the loss. Standard cases follow from direct differentiation.

Squared loss. For regression with $\ell(y, F) = \frac{1}{2}(y - F)^2$, $\partial\ell/\partial F = -(y - F)$, so the pseudo-residual is the ordinary residual:

$$r_i^{(m)} = y_i - \hat{f}^{(m-1)}(x_i).$$

Squared-loss gradient boosting recovers Definition 5.33 exactly: line search at round m gives $\hat{\nu}_m = 1$, since the residual fit is already on the right scale.

Absolute loss. For robust regression with $\ell(y, F) = |y - F|$, $\partial\ell/\partial F = -\text{sign}(y - F)$ (with $\text{sign}(0) := 0$ at the kink), so

$$r_i^{(m)} = \text{sign}(y_i - \hat{f}^{(m-1)}(x_i)) \in \{-1, 0, 1\}.$$

Each tree fits the sign of the residual: $+1$ for rows the ensemble underpredicts, -1 for rows it overpre-

dicts. The line-search step then sets each leaf value of $\hat{h}_m$ to the median of the original residuals within the leaf (the median minimizes absolute loss over a constant predictor), recovering the median-tree variant of Friedman (2001).

Huber loss. For robust regression, the Huber loss with threshold $\delta > 0$ is

$$\ell_\delta(y, F) = \begin{cases} \frac{1}{2}(y - F)^2, & |y - F| \leq \delta, \\ \delta(|y - F| - \delta/2), & |y - F| > \delta. \end{cases}$$

The two branches and their first derivatives match at $|y - F| = \delta$, so ℓ_δ is C^1 everywhere. Differentiating,

$$\frac{\partial \ell_\delta}{\partial F} = \begin{cases} -(y - F), & |y - F| \leq \delta, \\ -\delta \operatorname{sign}(y - F), & |y - F| > \delta, \end{cases}$$

so the pseudo-residual is the residual clipped to $[-\delta, \delta]$:

$$r_i^{(m)} = \operatorname{sign}(y_i - \hat{f}^{(m-1)}(x_i)) \cdot \min(|y_i - \hat{f}^{(m-1)}(x_i)|, \delta).$$

Rows within δ of the current prediction contribute their full residual (squared-loss regime); rows beyond δ contribute the clipped magnitude $\pm\delta$ (absolute-loss regime). The threshold δ interpolates

between the two: $\delta \rightarrow \infty$ recovers squared loss, $\delta \rightarrow 0$ recovers absolute loss (up to scale). The line-search leaf value is the Huber M-estimator of in-leaf residuals, with no closed form; Friedman (2001) approximates it by initializing at the leaf-residual median and taking one Huber-Newton step.

Quantile loss. For quantile regression at level $\tau \in (0, 1)$, the asymmetric pinball loss is

$$\ell_\tau(y, F) = \begin{cases} \tau(y - F), & y \geq F, \\ (1 - \tau)(F - y), & y < F. \end{cases}$$

Differentiating (with the convention $\partial \ell_\tau / \partial F|_{y=F} := 0$),

$$\frac{\partial \ell_\tau}{\partial F} = \mathbb{1}\{y < F\} - \tau,$$

so the pseudo-residual is constant within each side of the current prediction:

$$r_i^{(m)} = \tau - \mathbb{1}\{y_i < \hat{f}^{(m-1)}(x_i)\} \in \{\tau, \tau - 1\}.$$

Rows above the current prediction contribute τ , rows below contribute $\tau - 1$. The line-search leaf value is the empirical τ -quantile of in-leaf residuals (the global minimizer of $\sum \ell_\tau$ over a constant predictor).

Setting $\tau = 0.5$ recovers absolute loss up to a factor of $1/2$; running boosters at $\tau \in \{0.05, 0.5, 0.95\}$ produces a quantile-regression triplet for prediction intervals.

Binomial deviance (logistic loss). For binary classification with $y \in \{0, 1\}$, the model predicts a raw score F that maps to probability $p = \sigma(F) = 1/(1 + e^{-F})$. The negative log-likelihood is

$$\ell(y, F) = -yF + \log(1 + e^F).$$

Differentiating: $\partial\ell/\partial F = -y + \sigma(F) = \sigma(F) - y$, so

$$r_i^{(m)} = y_i - \sigma(\hat{f}^{(m-1)}(x_i)) = y_i - \hat{p}_i^{(m-1)},$$

the difference between the observed label and the current predicted probability, with values in $[-1, 1]$. After fitting $\hat{h}_m$ to these pseudo-residuals, a single Newton step on the leaf-level binomial likelihood gives the closed-form leaf value

$$c_{\text{leaf}}^* = \frac{\sum_{i \in \text{leaf}} r_i^{(m)}}{\sum_{i \in \text{leaf}} \hat{p}_i^{(m-1)} (1 - \hat{p}_i^{(m-1)})},$$

since $\partial^2\ell/\partial F^2 = p(1 - p)$ (the Hessian's leaf-restricted Newton-Raphson update; Friedman, 2001).

Multinomial deviance. For K -class classification with $y \in \{1, \dots, K\}$, the model maintains a vector of K raw scores per row, $F = (F_1, \dots, F_K)$. Predictions are softmax probabilities $p_k = e^{F_k} / \sum_{k'} e^{F_{k'}}$. The negative log-likelihood is

$$\ell(y, F) = -F_y + \log \sum_{k=1}^K e^{F_k}.$$

Differentiating with respect to each F_k :

$$\frac{\partial \ell}{\partial F_k} = -\mathbb{1}\{y = k\} + \frac{e^{F_k}}{\sum_{k'} e^{F_{k'}}} = \sigma_k(F) - \mathbb{1}\{y = k\},$$

giving per-class pseudo-residuals

$$r_{i,k}^{(m)} = \mathbb{1}\{y_i = k\} - \sigma_k(\hat{F}_i^{(m-1)}).$$

Each round fits K trees $\hat{h}_{m,1}, \dots, \hat{h}_{m,K}$ (one per class) to the per-class pseudo-residuals; each per-class score updates independently, $\hat{F}_k^{(m)} := \hat{F}_k^{(m-1)} + \nu \hat{h}_{m,k}$. The leaf-value Newton step uses the multinomial Hessian; Friedman (2001) keeps only the K diagonal entries for tractability.

4.3 Shrinkage, subsampling, and early stopping

Boosting fits training risk arbitrarily well as M grows; the ensemble's effective complexity must be controlled. Modern implementations combine three regularizers.

Definition 5.37 (Regularizers for gradient boosting). Practical implementations of Definition 5.35 augment it with:

- **Shrinkage**: replace the round- m step size $\hat{\nu}_m$ with a fixed constant $\nu \in (0, 1]$.
- **Row subsampling**: at round m , fit $\hat{h}_m$ to a uniformly random subset of ηn rows, $\eta \in (0, 1]$.
- **Early stopping**: monitor validation loss as m grows; halt at the first rise (or plateau).

Shrinkage (ν). Smaller ν slows convergence (more rounds are needed to reach a given training loss) but produces a smoother ensemble that generalizes better. The $\nu \rightarrow 0$ limit corresponds to gradient flow on the empirical risk; $\nu = 1$ recovers Definition 5.35's line-search variant. Typical values $\nu \in [0.01, 0.3]$.

Row subsampling (η). Friedman’s stochastic gradient boosting (2002) reduces tree-to-tree correlation, lowering ensemble variance via the same mechanism as bagging (Lemma 5.24). Modern implementations also subsample columns per tree or per node (`xgboost`’s `colsample_bytree` and `colsample_bynode`, analogous to RF’s `mtry`; §5.3.3). Typical values $\eta \in [0.5, 1.0]$.

Early stopping. Mirrors CV-based α selection in cost-complexity pruning (§5.2.3): a held-out validation fold determines when the ensemble’s complexity has saturated. With small ν and a generous $M_{\max}$, validation-tracked early stopping is the standard mechanism for choosing M .

The three address distinct overfitting modes: shrinkage limits per-step magnitude, subsampling decorrelates the trees, and early stopping bounds total ensemble complexity. Standard practice combines all three, exploring $\nu \in \{0.01, 0.05, 0.1\}$ and $\eta \in \{0.5, 0.7, 1.0\}$ with $M_{\max}$ effectively infinite (controlled by early stopping).

5 XGBoost: Second-Order Boosting with Regularization

XGBoost (Chen and Guestrin, 2016) refines gradient boosting in two coupled ways: a regularized per-tree objective with explicit penalties on leaf count and leaf-weight magnitude, and a second-

order Taylor expansion that yields closed-form leaf weights and a split-gain criterion. The same mathematical core powers LightGBM (Ke et al., 2017) and CatBoost (Prokhorenkova et al., 2018).

5.1 Second-order Taylor expansion of the per-stage loss

Regularized per-stage objective. At round m , gradient boosting (Definition 5.35) chooses the next weak learner h_m to decrease the empirical risk. XGBoost augments the round- m objective with an explicit complexity penalty on the tree itself:

$$\mathcal{L}^{(m)}(h_m) := \sum_{i=1}^n \ell(y_i, \hat{F}_i^{(m-1)} + h_m(x_i)) + \Omega(h_m), \quad \Omega(h_m) := \gamma T_m + \frac{1}{2} \lambda \sum_{j=1}^{T_m} w_{m,j}^2,$$

where T_m is the number of leaves of h_m , $w_{m,j}$ is its j -th leaf value, and $\gamma, \lambda \geq 0$ are hyperparameters penalizing leaf count and squared leaf-weight magnitude.

Quadratic approximation. Expand the loss to second order around $\hat{F}_i^{(m-1)}$. Write

$$g_i := \left. \frac{\partial \ell(y_i, F)}{\partial F} \right|_{F=\hat{F}_i^{(m-1)}}, \quad H_i := \left. \frac{\partial^2 \ell(y_i, F)}{\partial F^2} \right|_{F=\hat{F}_i^{(m-1)}}$$

for the per-row gradient and Hessian (assumed positive in a neighborhood of the optimum). Then

$$\ell(y_i, \hat{F}_i^{(m-1)} + h_m(x_i)) \approx \ell(y_i, \hat{F}_i^{(m-1)}) + g_i h_m(x_i) + \frac{1}{2} H_i h_m(x_i)^2.$$

Dropping the constant $\sum_i \ell(y_i, \hat{F}_i^{(m-1)})$ (independent of h_m),

$$\tilde{\mathcal{L}}^{(m)}(h_m) = \sum_{i=1}^n [g_i h_m(x_i) + \frac{1}{2} H_i h_m(x_i)^2] + \gamma T_m + \frac{1}{2} \lambda \sum_{j=1}^{T_m} w_{m,j}^2.$$

$\tilde{\mathcal{L}}^{(m)}$ is quadratic in the leaf-value vector once h_m 's tree structure is fixed and admits a closed-form minimizer.

5.2 Closed-form leaf weights and split gain

Per-leaf statistics. A regression tree h_m with T leaves partitions $\mathcal{X}$ into regions $R_1, \dots, R_T$ and assigns leaf value w_j to R_j , so $h_m(x) = \sum_j w_j \mathbb{1}\{x \in R_j\}$. Let $I_j := \{i : x_i \in R_j\}$ be the rows reaching leaf j , and define

$$G_j := \sum_{i \in I_j} g_i, \quad H_j := \sum_{i \in I_j} H_i.$$

Substituting $h_m(x_i) = w_j$ for $i \in I_j$ and grouping by leaf,

$$\tilde{\mathcal{L}}^{(m)} = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T.$$

Lemma 5.38 (Closed-form optimal leaf weights). For a fixed tree structure (leaves $\{R_j\}$), the leaf weights minimizing $\tilde{\mathcal{L}}^{(m)}$ are

$$w_j^* = -\frac{G_j}{H_j + \lambda},$$

yielding the structure score

$$\tilde{\mathcal{L}}^{(m)*} = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T.$$

Proof. $\tilde{\mathcal{L}}^{(m)}$ is separable in the w_j . For each j , the per-leaf objective $G_j w + \frac{1}{2} (H_j + \lambda) w^2$ is a strictly convex quadratic in w (since $H_j + \lambda > 0$). Setting the derivative to zero: $G_j + (H_j + \lambda) w = 0$,

giving $w_j^* = -G_j/(H_j + \lambda)$. Substituting back,

$$G_j w_j^* + \frac{1}{2}(H_j + \lambda)(w_j^*)^2 = -\frac{G_j^2}{H_j + \lambda} + \frac{G_j^2}{2(H_j + \lambda)} = -\frac{G_j^2}{2(H_j + \lambda)}.$$

Summing over leaves and adding γT gives the structure score. □

The structure score is the XGBoost analogue of within-tree SSE for regression CART (Definition 5.7), combining the second-order Taylor fit with explicit regularization.

Theorem 5.39 (XGBoost split gain). Splitting a leaf L (statistics G_L, H_L) into children L_l, L_r (statistics G_l, H_l, G_r, H_r with $G_L = G_l + G_r, H_L = H_l + H_r$) decreases the structure score by

$$\text{Gain} = \frac{1}{2} \left[\frac{G_l^2}{H_l + \lambda} + \frac{G_r^2}{H_r + \lambda} - \frac{G_L^2}{H_L + \lambda} \right] - \gamma.$$

A split is performed iff $\text{Gain} > 0$.

Proof. Before the split, leaf L contributes $-\frac{1}{2}G_L^2/(H_L + \lambda) + \gamma$ to $\tilde{\mathcal{L}}^{(m)*}$ (one γ per leaf). After the split, the two children together contribute $-\frac{1}{2}[G_l^2/(H_l + \lambda) + G_r^2/(H_r + \lambda)] + 2\gamma$. The change in structure score is

$$\Delta\tilde{\mathcal{L}}^{(m)*} = -\frac{1}{2}\left[\frac{G_l^2}{H_l + \lambda} + \frac{G_r^2}{H_r + \lambda} - \frac{G_L^2}{H_L + \lambda}\right] + \gamma.$$

Gain is the negative of this change. The split improves the structure score only when $\text{Gain} > 0$. $\square$

Recovery of CART's SSE gain. For squared loss with no regularization ($\lambda = \gamma = 0$): $g_i = -(y_i - \hat{F}_i^{(m-1)}) = -r_i^{(m)}$ and $H_i = 1$, so $G_j = -\sum_{i \in I_j} r_i^{(m)} = -n_j \bar{r}_j$ and $H_j = n_j$. Then $G_j^2/(H_j + \lambda) = n_j \bar{r}_j^2$, and $\text{Gain} = \frac{1}{2}[n_l \bar{r}_l^2 + n_r \bar{r}_r^2 - n_L \bar{r}_L^2]$. The algebraic identity in Lemma 5.9's proof gives $\text{Gain} = \frac{1}{2} \cdot n_l n_r (\bar{r}_l - \bar{r}_r)^2 / n_L$, the SSE gain of Lemma 5.9 up to a factor of 1/2. XGBoost on squared loss is CART on residuals, with the L1/L2 regularizers turned off.

5.3 Regularization knobs and modern variants

The second-order framework exposes structural hyperparameters that complement the §5.4.3 regularizers.

λ (L2 leaf-weight penalty). Appears in the denominator of $w_j^* = -G_j/(H_j + \lambda)$ and in every term of the structure score. Larger λ shrinks every leaf weight toward zero and lowers the per-leaf structure-score contribution, making splits less attractive. Acts as ridge regularization on the boosted ensemble's per-leaf adjustments.

α (L1 leaf-weight penalty). Adds $\alpha \sum_j |w_j|$ to Ω . The optimal leaf weight becomes the soft-thresholded $w_j^* = -\text{soft}_\alpha(G_j)/(H_j + \lambda)$ with $\text{soft}_\alpha(x) = \text{sign}(x) \cdot \max(|x| - \alpha, 0)$ (compare Unit 2 §2.3.3 for soft-thresholding under Lasso). Promotes sparse leaf weights: leaves with $|G_j| < \alpha$ are zeroed out.

γ (per-leaf complexity penalty). Acts as a minimum-gain threshold on splits. By Theorem 5.39, a split is accepted iff $\frac{1}{2}[G_l^2/(H_l + \lambda) + G_r^2/(H_r + \lambda) - G_L^2/(H_L + \lambda)] > \gamma$. Larger γ blocks marginal splits, producing shallower trees.

Depth and min_child_weight. `max_depth` caps tree depth directly, bounding model flexibility per tree. `min_child_weight` requires $H_l, H_r \geq w_{\min}$ for any candidate split. For squared loss $H_i = 1$,

so this is a minimum sample-count constraint per child; for other losses it weights samples by their Hessian (the per-row second-order information), preventing splits that isolate confidently-classified rows.

Carry-overs from §5.4.3. `eta` is XGBoost's name for the shrinkage ν ; `subsample` is the row fraction η ; `colsample_bytree` and `colsample_bynode` extend RF-style feature subsampling (§5.3.3) to boosting. Validation-tracked early stopping picks M .

LightGBM and CatBoost

LightGBM (Ke et al., 2017) bins continuous features into integer histograms with per-bin gradient and Hessian sums, plus gradient-based one-sided sampling (GOSS: keep large-gradient rows) and exclusive feature bundling (EFB: pack sparse mutually-exclusive features). CatBoost (Prokhorenkova et al., 2018) prevents target leakage in mean target encoding via **ordered boosting**: each row's target statistic uses only rows preceding it in a random permutation. Both build on Theorem 5.39's objective; the differences lie in the splitting search, sampling, and categorical handling, not the objective itself.

5.4 Practical extensions to XGBoost-family boosting

XGBoost, LightGBM, and CatBoost share Theorem 5.39's gain formula but diverge in candidate-split search, missing-value routing, and categorical handling. The first three extensions below appear in every production tabular pipeline; the rest are situational.

Histogram-based splitting. Exact splitting sorts each feature in a node and scores all $n_I - 1$ cuts in $O(n_I \log n_I)$. Histogram methods bin each feature once into $B \approx 256$ integer bins and consider only bin-boundary cuts; per-feature per-node cost falls to $O(n_I + B)$. XGBoost's `tree_method="hist"` is the default; LightGBM is histogram-only. The gain formula is unchanged.

Lemma 5.40 (Histogram split gain). Partition feature $X^{(j)}$ into bins $b = 1, \dots, B$ with per-bin sums

$$G^{(b)} := \sum_{i \in I_L, x_i^{(j)} \in b} g_i, \quad H^{(b)} := \sum_{i \in I_L, x_i^{(j)} \in b} H_i.$$

For each boundary b , the cumulative sums $G_l = \sum_{b' \leq b} G^{(b')}$, $H_l = \sum_{b' \leq b} H^{(b')}$ substitute into Theorem 5.39. All $B - 1$ gains follow in $O(B)$ after one $O(n_I)$ pass to populate $\{G^{(b)}, H^{(b)}\}$.

Proof. Cumulative bin sums equal the row sums up to a boundary, so plugging them into Theorem 5.39 yields the gain at that boundary. Cumulative sums update incrementally in b . $\square$

Missing-value handling. XGBoost's **default-direction** trick (Chen and Guestrin, 2016, §3.4) learns the routing of missing rows instead of imputing in preprocessing. For each candidate split, scan non-missing rows as usual; route all missing rows to the side that yields the larger gain.

Lemma 5.41 (Default-direction split gain). Let (G_l°, H_l°) , (G_r°, H_r°) be the left/right sums over non-missing rows on the candidate feature, and (G_M, H_M) the sums over missing rows in I_L . The optimal default-direction gain is $\max\{\text{Gain}_l, \text{Gain}_r\}$ with

$$\text{Gain}_l := \frac{1}{2} \left[\frac{(G_l^\circ + G_M)^2}{H_l^\circ + H_M + \lambda} + \frac{(G_r^\circ)^2}{H_r^\circ + \lambda} - \frac{G_L^2}{H_L + \lambda} \right] - \gamma,$$

and Gain_r symmetric (sending (G_M, H_M) to the right child).

Proof. Fix a default direction; the child sums are as stated, so the gain is an instance of Theorem 5.39. The optimum over the two directions is the larger of the two. $\square$

Cost: two evaluations of Theorem 5.39 per candidate, plus one $O(|I_L|)$ pass to accumulate (G_M, H_M) .

Native categorical handling. One-hot encoding a K -level categorical creates K binary features and forces axis-aligned splits to peel one level at a time, underfitting when the optimal split groups several levels into one leaf. The three majors handle categoricals natively:

Library	Strategy
LightGBM	Sort levels by $G_k/(H_k + \lambda)$ and scan as continuous (Fisher's optimal partitioning).
CatBoost	Ordered target statistics: per-row target encoding using only rows preceding it in a random permutation (the same device as ordered boosting).
XGBoost (≥ 2.0)	Sort levels by gradient ratio and scan as continuous (<code>enable_categorical=TRUE</code>).

Sorting by gradient ratio reduces a K -level categorical to a continuous scan, recovering the $O(B)$ per-node cost of Lemma 5.40. CatBoost's permutation alone defends against target leakage in the

encoding itself; the sort-by-gradient strategies in LightGBM and XGBoost still need an external CV-aware split for safe target encoding.

Further extensions

Monotonic constraints (`monotone_constraints`) reject splits whose optimal leaf weights w_j^* violate user-specified per-feature monotonicity; required in regulated settings such as credit scoring and insurance pricing. **DART** (Rashmi and Gilad-Bachrach, 2015) replaces shrinkage with random tree dropout during boosting; available as `booster="dart"` in XGBoost and natively in LightGBM. **NGBoost** (Duan et al., 2020) predicts a parametric conditional distribution rather than a point estimate, supplying calibrated uncertainty for downstream decisions; implemented in the separate `ngboost` package. **Survival objectives** (`objective="survival:cox"`, `"survival:aft"`) extend XGBoost to right-censored time-to-event outcomes; standard in biostatistics and insurance loss modeling.

6 Implementation Tour

The theory of §§2–5 maps onto four production libraries (`rpart`, `ranger`, `xgboost`, the `tidymodels` unified API) and a from-scratch CART. All chunks use `modeldata::credit_data` (13 predictors, mixed numeric and categorical) so library comparisons hold the data fixed.

6.1 CART with `rpart`

`rpart` (Therneau and Atkinson) implements CART with cost-complexity pruning, defaulting to Gini for classification and residual SSE for regression (§§2.1–2.2). Grow to a permissive size (`cp = 0`), then prune at the complexity selected from the built-in 10-fold CV table (`xval = 10`); pick the `xerror` minimizer or the smaller 1-SE tree.

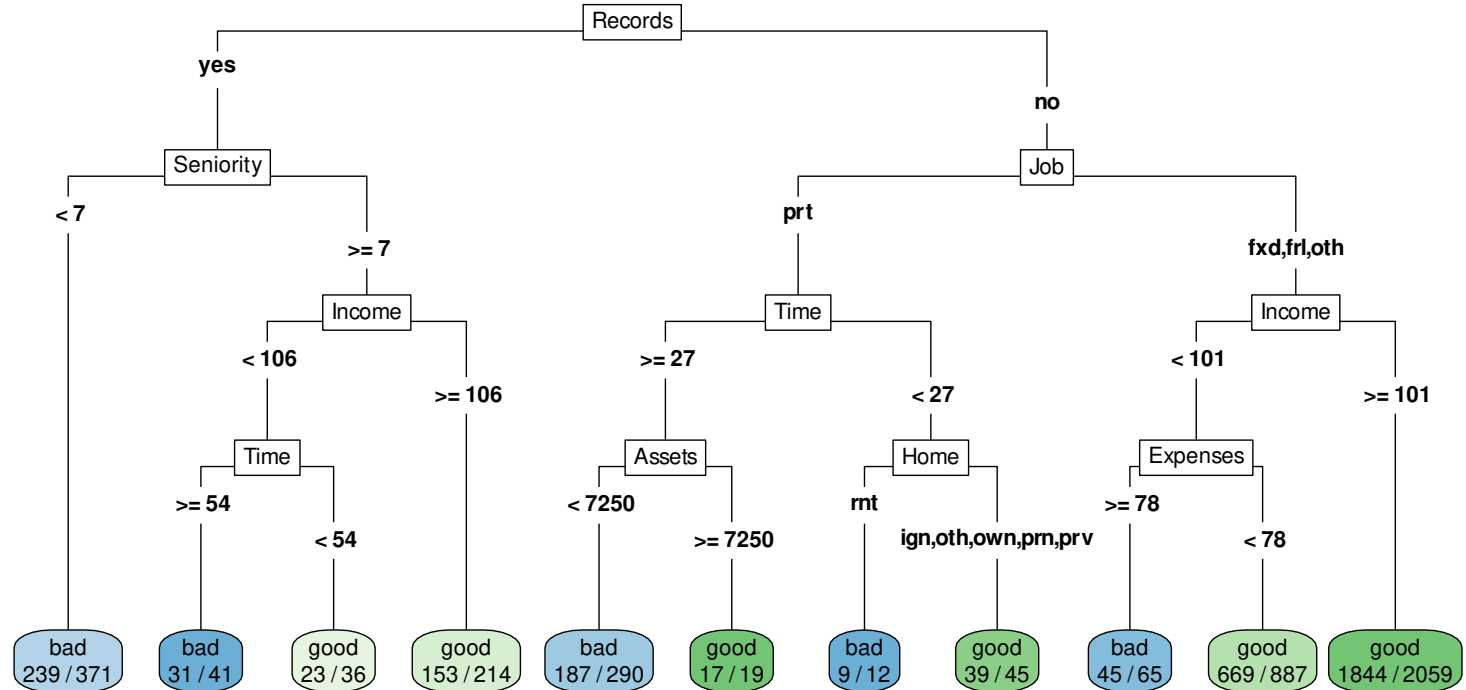
```
1 library(rpart); library(rpart.plot)
2 data("credit_data", package = "modeldata")
3 credit <- na.omit(credit_data)
4
5 set.seed(1)
6 fit_full <- rpart(Status ~ ., data = credit, method = "class",
7                   control = rpart.control(cp = 0, xval = 10, maxdepth = 4))
8
```

```
9 # Cost-complexity table: each row is a pruned subtree; xerror is  
10 # cross-validated relative error (root error normalized to 1)  
11 cptab <- fit_full$cptable  
12 print(cptab)
```

##		CP	nsplit	rel error	xerror	xstd
## 1	0.052144250		0	1.0000000	1.0000000	0.02696429
## 2	0.040935673		2	0.8957115	0.9805068	0.02678865
## 3	0.026315789		3	0.8547758	0.9493177	0.02649784
## 4	0.014619883		4	0.8284600	0.8762183	0.02576679
## 5	0.012183236		5	0.8138402	0.8625731	0.02562230
## 6	0.010721248		7	0.7894737	0.8635478	0.02563271
## 7	0.009746589		8	0.7787524	0.8499025	0.02548578
## 8	0.005847953		9	0.7690058	0.8430799	0.02541132
## 9	0.001461988		10	0.7631579	0.8333333	0.02530377
## 10	0.000000000		12	0.7602339	0.8391813	0.02536847

```
1 # Prune at the complexity minimizing xerror  
2 cp_best <- cptab[which.min(cptab[, "xerror"]), "CP"]  
3 fit_pruned <- prune(fit_full, cp = cp_best)  
4
```

```
5 rpart.plot(fit_pruned, type = 5, extra = 2, faclen = 3)
```



cp normalization and the 1-SE rule

rpart's **cp** is the cost-complexity penalty α divided by the root risk $R(T_0)$ (§2.3), and **xerror** reports cross-validated risk on the same normalization, so both columns are dimensionless. The 1-SE rule picks the simplest tree with **xerror** within **xstd** of the minimum, trading a small upward bias in CV error for a substantially smaller tree; **xstd** sits in the same table column for the lookup.

6.2 Random Forest with ranger

ranger (Wright and Ziegler, 2017) is a C++ Random Forest implementation with native OOB error (Lemma 5.22) and permutation importance (§3.4). The two primary hyperparameters are **mtry** (per-split feature subsample size m , §3.3) and **num.trees** (ensemble size B , default 500); OOB error stabilizes after a few hundred trees and acts as a free held-out estimate.

```
1 library(ranger)
2 data("credit_data", package = "modeldata")
3 credit <- na.omit(credit_data)
4
5 set.seed(1)
```

```

6 fit_rf <- ranger(Status ~ ., data = credit, num.trees = 500,
7                 mtry = floor(sqrt(ncol(credit) - 1)),
8                 importance = "permutation")
9
10 fit_rf$prediction.error           # OOB classification error

```

```
## [1] 0.1995543
```

```

1 sort(importance(fit_rf), decreasing = TRUE)[1:6]  # top permutation-importance features

```

```

##      Records      Amount      Job      Income Seniority      Assets
## 0.02170001 0.02046274 0.01986394 0.01798960 0.01760726 0.01170567

```

Tuning mtry

The defaults $m = \lceil \sqrt{p} \rceil$ (classification) and $m = \lfloor p/3 \rfloor$ (regression) (§3.3) work well as starting points; sweep $m \in \{2, 4, 8, \dots, p\}$ and select on OOB error for further gains. Larger m reduces tree-to-tree decorrelation, returning toward bagging's variance reduction; smaller m trades single-tree quality for stronger averaging.

6.3 Gradient boosting with xgboost

xgboost implements §5's regularized second-order objective (Theorem 5.39) with histogram-based splitting (Lemma 5.40) as the default. The `data.frame` is passed directly: factor columns become categorical (§5.4) and **NA** values trigger Lemma 5.41's default-direction routing, both without any preprocessing step. Standard pattern: fix conservative defaults (`eta = 0.05`, `max_depth = 6`, `subsample = 0.8`), call `xgb.cv` with early stopping to pick the number of trees M , and refit on the full training data.

```
1 library(xgboost)
2 data("credit_data", package = "modeldata")
3 sum(is.na(credit_data))                # 455 NAs across 4454 rows
```

```
## [1] 455
```

```
1 # Pass the data.frame directly: factors become categorical and NA values
2 # route via the default-direction trick. No na.omit, no model.matrix.
3 X <- credit_data[, setdiff(names(credit_data), "Status")]
4 y <- as.integer(credit_data$Status == "good")      # 1 = good, 0 = bad
5
6 set.seed(1)
```

```
7 dtrain <- xgb.DMatrix(data = X, label = y)
8 params <- list(objective = "binary:logistic", eval_metric = "logloss",
9               tree_method = "hist", eta = 0.05, max_depth = 6,
10              subsample = 0.8, colsample_bytree = 0.8,
11              lambda = 1, gamma = 0)
12
13 cv <- xgb.cv(params = params, data = dtrain, nrounds = 500, nfold = 5,
14            early_stopping_rounds = 20, verbose = 0)
15 M_best <- cv$early_stop$best_iteration           # selected by validation logloss
16
17 fit <- xgb.train(params = params, data = dtrain, nrounds = M_best)
```

Reading the CV log

`xgb.cv` tracks mean and standard deviation of training and validation logloss per round; `best_iteration` is the validation-minimum round. `early_stopping_rounds = 20` halts training after 20 consecutive rounds without validation improvement. The final `xgb.train` refits on `dtrain` for $M = M_best$ rounds; the validation curve picked M but is not the final-fit metric.

6.4 Tidymodels integration

`parsnip` unifies the three libraries: `decision_tree()` (`rpart` engine), `rand_forest()` (`ranger` engine), `boost_tree()` (`xgboost` engine; `lightgbm` and `catboost` via `bonsai`). A shared `recipe` handles preprocessing in-fold; `tune_grid` sweeps hyperparameters by cross-validation.

```
1 library(tidymodels); library(xgboost)
2 data("credit_data", package = "modeldata")
3 credit <- na.omit(credit_data)
4
5 set.seed(1)
6 split <- initial_split(credit, prop = 0.8, strata = Status)
7 train <- training(split); test <- testing(split)
8 folds <- vfold_cv(train, v = 5, strata = Status)
9
10 rec <- recipe(Status ~ ., data = train) |>
11   step_dummy(all_nominal_predictors(), one_hot = TRUE)
12
13 mod <- boost_tree(trees = tune(), learn_rate = tune(),
14                  tree_depth = 6, sample_size = 0.8) |>
15   set_engine("xgboost") |> set_mode("classification")
```

```
16
17 wf  <- workflow() |> add_recipe(rec) |> add_model(mod)
18
19 grid <- grid_regular(trees(range = c(50, 500)),
20                      learn_rate(range = c(-2, -1)), levels = c(5, 4))
21
22 res <- tune_grid(wf, resamples = folds, grid = grid,
23                 metrics = metric_set(roc_auc))
24
25 final_wf  <- finalize_workflow(wf, select_best(res, metric = "roc_auc"))
26 final_res <- last_fit(final_wf, split)
27 collect_metrics(final_res)
```

```
## # A tibble: 3 x 4
```

```
##   .metric      .estimator .estimate .config
##   <chr>        <chr>         <dbl> <chr>
## 1 accuracy    binary           0.802 pre0_mod0_post0
## 2 roc_auc      binary           0.842 pre0_mod0_post0
## 3 brier_class binary           0.136 pre0_mod0_post0
```

The in-fold preprocessing guarantee

`step_dummy` fits its level mapping on each training fold and applies it to the validation fold; `last_fit` is the only call that touches the test split. Encoding outside the workflow (e.g., `model.matrix` on the full data before `initial_split`) leaks test-fold information and inflates CV performance, the same trap as Lasso standardization (Unit 2 §2.7.2). `step_dummy` is needed here because `parsnip`'s `xgboost` engine routes data through a numeric matrix and forfeits native categorical handling; for the native path call `xgb.train` directly (§6.3), or use `bonsai`'s `lightgbm/catboost` engines, which preserve factors.

Beyond grid search

`tune_grid` evaluates a fixed Cartesian product. `tune::tune_bayes` (Gaussian-process surrogate over hyperparameters) and `finetune::tune_race_anova` (ANOVA racing prunes weak combinations after a few resamples) drop in for `tune_grid` without changing the workflow.

6.5 From-scratch CART

A from-scratch implementation makes Definition 5.6's recursion concrete: a Gini computation (Definition 5.11), a per-node best-split scan over (j, v) pairs, and a recursive grower with depth and minimum-node-size stopping rules.

```
1 gini <- function(y) {  
2   p <- prop.table(table(y))  
3   1 - sum(p^2)  
4 }  
5  
6 best_split <- function(X, y, min_n) {  
7   n <- length(y); best <- list(gain = 0)  
8   g_parent <- gini(y)  
9   for (j in seq_len(ncol(X))) {  
10    vals <- sort(unique(X[, j]))  
11    for (v in vals[-length(vals)]) { # candidate threshold  
12      left <- X[, j] <= v  
13      if (sum(left) < min_n || sum(!left) < min_n) next  
14      gain <- g_parent -  
15        (sum(left) / n) * gini(y[ left]) -
```

```
16         (sum(!left) / n) * gini(y[!left])
17         if (gain > best$gain) best <- list(gain = gain, j = j, v = v)
18     }
19 }
20 best
21 }
22
23 build_tree <- function(X, y, depth = 0, max_depth = 4, min_n = 5) {
24     if (depth >= max_depth || length(unique(y)) == 1)
25         return(list(leaf = TRUE, pred = names(which.max(table(y)))))
26     s <- best_split(X, y, min_n)
27     if (is.null(s$j) || s$gain == 0)
28         return(list(leaf = TRUE, pred = names(which.max(table(y)))))
29     left <- X[, s$j] <= s$v
30     list(leaf = FALSE, j = s$j, v = s$v,
31          l = build_tree(X[ left, , drop = FALSE], y[ left], depth + 1, max_depth, min_n),
32          r = build_tree(X[!left, , drop = FALSE], y[!left], depth + 1, max_depth, min_n))
33 }
34
35 predict_tree <- function(tree, x) {
36     if (tree$leaf) return(tree$pred)
```

```
37   if (x[tree$j] <= tree$v) predict_tree(tree$l, x) else predict_tree(tree$r, x)
38 }
39
40 # Smoke test on credit_data (numeric features only)
41 data("credit_data", package = "modeldata")
42 credit <- na.omit(credit_data)
43 num_cols <- sapply(credit, is.numeric)
44 X <- as.matrix(credit[, num_cols]); y <- credit$Status
45
46 set.seed(1)
47 tree <- build_tree(X, y, max_depth = 4, min_n = 20)
48 pred <- apply(X, 1, function(x) predict_tree(tree, x))
49 mean(pred == y) # training accuracy
```

```
## [1] 0.7751919
```

What the toy implementation omits

The scan re-sorts each feature's unique values at every node ($O(n_I \log n_I)$ per feature per node), and `predict_tree` loops row-by-row through R's `apply`. Production libraries pre-sort each fea-

ture once and reuse sorted indices for splits (§3.3, $O(n_I)$ per feature per node), and dispatch the entire test-set index array down the tree in one vectorized predict pass. Other omissions: regression splits (Definition 5.8), surrogate splits for missing values (**rpart**'s standard mechanism), and cost-complexity pruning (§2.3). The grower's purpose is to make Definition 5.6's recursion legible, not to replace **rpart**.

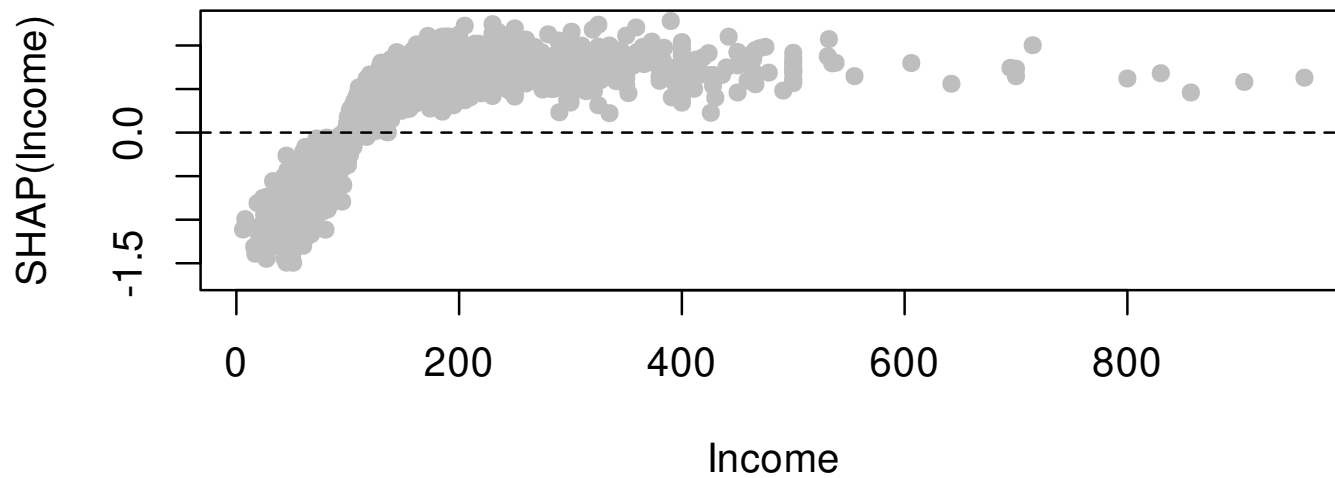
6.6 Explaining boosted trees with SHAP

TreeSHAP (§3.5) is implemented natively in **xgboost** via `predcontrib = TRUE`: `predict` returns an $n \times (p + 1)$ matrix whose first p columns are per-row SHAP values for each feature and whose last column is the bias term. Aggregations of this matrix produce the standard interpretability views.

```
1 # Reuse fit and dtrain from the xgboost-tour chunk; recover X
2 data("credit_data", package = "modeldata")
3 X_xgb <- credit_data[, setdiff(names(credit_data), "Status")]
4
5 shap <- predict(fit, dtrain, predcontrib = TRUE)
6 shap <- shap[, -ncol(shap), drop = FALSE]           # drop bias column
7 sort(colMeans(abs(shap)), decreasing = TRUE)[1:6]   # global importance
```

```
##      Income Seniority  Records    Amount    Assets      Job  
## 0.5792341 0.4816023 0.4694750 0.4635825 0.3397866 0.2422442
```

```
1 # Dependence plot: SHAP(Income) versus Income  
2 plot(X_xgb$Income, shap[, "Income"], pch = 16, col = "grey",  
3      xlab = "Income", ylab = "SHAP(Income)")  
4 abline(h = 0, lty = 2)
```



Naive Shapley vs. TreeSHAP

Naive Shapley evaluates 2^p feature subsets per row; TreeSHAP exploits the tree structure (§3.5) to compute the same expected-conditional values in $O(\sum_t L_t \cdot D_t^2)$ across the ensemble, where L_t and D_t are tree t 's leaf count and depth. The R packages **treeshap** (model-agnostic; supports **xgboost**, **lightgbm**, **catboost**, **ranger**) and **shapviz** (visualization layer) wrap this functionality.

6.7 Practitioner's cheat sheet

The hyperparameter discussions of §§4.3–5.3 produce a long list of knobs. The tour above shows their syntax; this section gives the order in which to reach for them.

Picking the family

Family	When to choose
Single CART (rpart)	The model itself is the deliverable: a transparent clinical or regulatory review. Accept lower prediction accuracy for interpretability.
Random Forest (ranger)	Strong baseline with minimal tuning. OOB error estimate; the default starting point when there is no other information to do otherwise.
Gradient boosting (xgboost , lightgbm , catboost)	Predictive accuracy is the priority and there is budget for a parameter sweep. Pair early stopping with a small number of trees.

Tuning priority when boosting overfits

Train loss falls well below validation loss; address in this order:

1. Reduce **eta** (e.g., $0.3 \rightarrow 0.05$) and let early stopping pick more rounds.
2. Cap **max_depth** (e.g., $6 \rightarrow 4$) and raise **min_child_weight** (e.g., $1 \rightarrow 5-10$).

3. Subsample rows (`subsample` = 0.7–0.9) and columns (`colsample_bytree` = 0.5–0.8).
4. Increase the L2 leaf-weight penalty `lambda` (e.g., $1 \rightarrow 5$); only after that reach for L1 (`alpha`) or the split-gain threshold (`gamma`).

The first three items handle most overfitting; `lambda` / `alpha` / `gamma` are second-line.

7 Pitfalls and Misapplications

Each entry pairs a misuse with a diagnostic and a runnable demo. Importance metrics, extrapolation, early stopping, encoding, and class imbalance dominate the list because uncalibrated splits and ensemble averaging mask the failure modes from casual inspection.

7.1 High-cardinality bias in MDI

MDI (mean decrease in impurity, §3.4) inflates the importance of features with many distinct values: each extra candidate split gives the feature another chance to find an apparent reduction, even when the feature is independent of the response. MDI is the default in `ranger(..., importance = "impurity")` and `randomForest::importance(..., type = 2)`, hiding the bias from tutorial readers.

```
1 library(ranger); set.seed(1)
2 n <- 500
3 df <- data.frame(
4   x_binary      = sample(0:1, n, replace = TRUE),
5   x_continuous  = runif(n),           # 500 distinct values
6   y             = rnorm(n)           # independent of both
7 )
8 fit_imp <- ranger(y ~ ., data = df, importance = "impurity")
9 fit_perm <- ranger(y ~ ., data = df, importance = "permutation")
10 rbind(MDI = importance(fit_imp), Permutation = importance(fit_perm))
```

```
##              x_binary x_continuous
## MDI          2.78774534 50.607562506
## Permutation -0.01638927 -0.005571546
```

MDI inflates `x_continuous` (≈ 50) over `x_binary` (≈ 3) although both are independent of y ; permutation returns near-zero for both.

Diagnostic. Compare MDI against permutation.

Fix. Use permutation importance as the primary metric.

7.2 Correlated features dilute importance

When two near-identical features sit in the model, both MDI and permutation importance split the credit between them; either alone gets the full signal.

```
1 library(ranger); set.seed(1)
2 n <- 500
3 x1 <- rnorm(n); x2 <- x1 + rnorm(n, sd = 0.05)           # near-copy
4 y <- 2 * x1 + rnorm(n)
5 both <- ranger(y ~ ., data = data.frame(y, x1, x2), importance = "permutation")
6 alone <- ranger(y ~ ., data = data.frame(y, x1), importance = "permutation")
7 importance(both); importance(alone)
```

```
##          x1          x2
## 3.996198 2.951366
##          x1
## 7.56827
```

x_1 alone scores ≈ 7.6 ; with x_2 added, the credit splits to 4.0 and 3.0.

Diagnostic. Drop one of a near-correlated pair and refit; a jump in the survivor's importance signals shared credit.

Fix. Cluster correlated features and report a grouped permutation, or drop the redundant copy before fitting.

7.3 ID-like columns rank top in importance

A near-unique column (a row identifier, a timestamp at second resolution, an account number) lets the tree split rows individually until every leaf is pure. The training fit looks excellent and the column climbs the importance ranking, but the tree has memorized rows rather than learned a pattern.

```
1 library(ranger); set.seed(1)
2 data("credit_data", package = "modeldata")
3 credit <- na.omit(credit_data)
4 credit$row_id <- seq_len(nrow(credit)) # injected near-unique column
5 fit <- ranger(Status ~ ., data = credit, importance = "impurity")
6 sort(importance(fit), decreasing = TRUE)[1:5]
```

```
##      Income      Amount      Price Seniority      row_id
## 191.2607 157.0593 154.9004 153.8884 149.3941
```

`row_id` appears in the top five alongside genuine predictors such as **Income** and **Amount**, despite carrying no information.

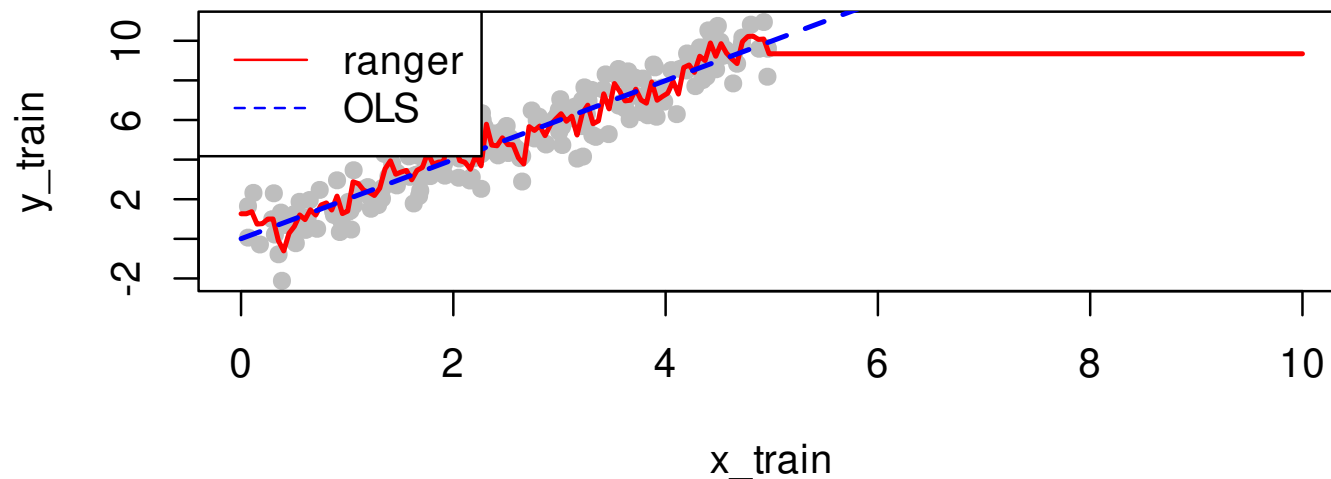
Diagnostic. Any top-ranked feature whose ranking is unreasonable in domain terms.

Fix. Remove identifiers before fitting; the modeling frame should contain only features whose values are reproducible at prediction time.

7.4 Random Forests extrapolate poorly

Tree predictions are constant within each leaf, so any test prediction is a convex combination of training-leaf averages. For a test point beyond the training range, every relevant leaf is the boundary leaf, and the prediction flatlines.

```
1 library(ranger); set.seed(1)
2 x_train <- runif(200, 0, 5)
3 y_train <- 2 * x_train + rnorm(200)
4 x_test  <- seq(0, 10, length.out = 200)
5 rf  <- ranger(y ~ x, data = data.frame(y = y_train, x = x_train))
6 ols <- lm(y_train ~ x_train)
7 plot(x_train, y_train, xlim = c(0, 10), pch = 16, col = "grey")
8 lines(x_test, predict(rf, data.frame(x = x_test))$predictions, col = "red", lwd = 2)
9 lines(x_test, predict(ols, data.frame(x_train = x_test)), col = "blue", lwd = 2, lty
10 legend("topleft", c("ranger", "OLS"), col = c("red", "blue"), lty = c(1, 2))
```



The RF curve is flat for $x > 5$ at the maximum-leaf training prediction; OLS continues linearly.

Diagnostic. Range-check test inputs against the training range before deploying.

Fix. Monitor input range; if inputs systematically extend beyond training, supplement with a para-

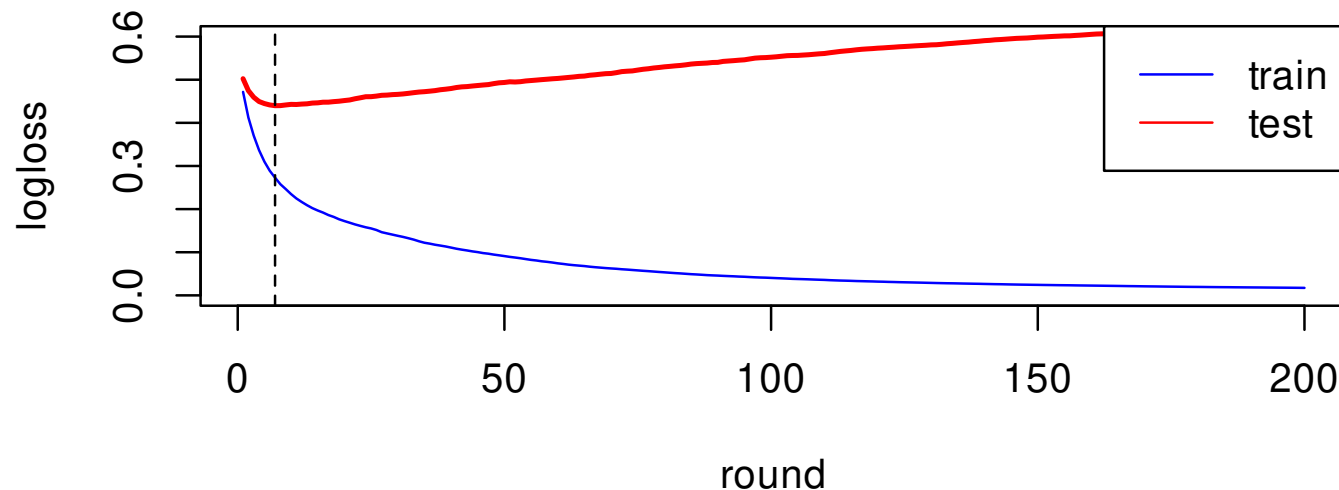
metric model or restrict the deployment scope.

7.5 Boosting overfits without early stopping

Gradient boosting reduces training loss monotonically by construction: every new tree fits the residual gradient. Without a held-out validation curve, the iteration count M has no upper bound and the model overfits.

```
1 library(xgboost); set.seed(1)
2 data("credit_data", package = "modeldata")
3 credit <- na.omit(credit_data)
4 X <- model.matrix(Status ~ . - 1, data = credit)
5 y <- as.integer(credit$Status == "good")
6 dtrain <- xgb.DMatrix(X, label = y)
7 cv <- xgb.cv(params = list(objective = "binary:logistic", eval_metric = "logloss",
8                             eta = 0.3, max_depth = 8),
9              data = dtrain, nrounds = 200, nfold = 5, verbose = 0)
10 log <- cv$evaluation_log
11 plot(log$iter, log$train_logloss_mean, type = "l", col = "blue", ylim = c(0, 0.6),
12      xlab = "round", ylab = "logloss")
13 lines(log$iter, log$test_logloss_mean, col = "red", lwd = 2)
14 abline(v = which.min(log$test_logloss_mean), lty = 2)
```

```
15 legend("topright", c("train", "test"), col = c("blue", "red"), lty = 1)
```



Train logloss falls toward zero; test logloss bottoms out near round 7 and then rises.

Diagnostic. The validation curve.

Fix. `early_stopping_rounds` on `xgb.cv` (§6.3) selects M at the validation minimum.

7.6 Trees do not need scaling

Splits are evaluated by impurity reduction at each candidate threshold (§§2.1–2.2); rescaling a feature linearly preserves the order of its values, so the same splits emerge with rescaled thresholds. `step_normalize` in a tree workflow is harmless but useless; spend the time on encoding and imputation instead.

```
1 library(rpart); set.seed(1)
2 n <- 200
3 df <- data.frame(x1 = runif(n, 0, 1),
4                 x2 = runif(n, 0, 1000),           # 1000x scale gap
5                 y  = factor(sample(c("a", "b"), n, replace = TRUE)))
6 fit_raw <- rpart(y ~ x1 + x2, data = df, control = rpart.control(cp = 0.01))
7 df$x1 <- as.numeric(scale(df$x1)); df$x2 <- as.numeric(scale(df$x2))
8 fit_sc  <- rpart(y ~ x1 + x2, data = df, control = rpart.control(cp = 0.01))
9 identical(fit_raw$frame$var, fit_sc$frame$var)    # same split sequence
```

```
## [1] TRUE
```

7.7 One-hot vs native categorical handling

A categorical with K levels under one-hot encoding produces K binary columns; each split tests one level against the rest, peeling levels one at a time. Native handling sorts levels by mean response (§5.4) and isolates the optimal subset in one split.

```
1 library(ranger); set.seed(1)
2 n <- 1000
3 city <- factor(sample(paste0("c", 1:20), n, replace = TRUE))
4 y <- (as.integer(city) %% 2) + rnorm(n, sd = 0.1) # even-indexed cities are signal
5 df_f <- data.frame(y = y, city = city)
6 df_o <- data.frame(y = y, model.matrix(~ city - 1, df_f))
7 oh <- ranger(y ~ ., data = df_o, num.trees = 200)
8 nat <- ranger(y ~ ., data = df_f, num.trees = 200,
9               respect.unordered.factors = "order") # sort by mean response
10 c(one_hot = oh$prediction.error, native = nat$prediction.error)
```

```
##      one_hot      native
## 0.01843753 0.01116324
```

Native handling reduces OOB MSE by roughly 40% (0.011 vs 0.018): one split isolates the relevant level subset, while one-hot needs many sparse splits for the same partition.

Diagnostic. Count distinct levels per categorical; for $K \gtrsim 10$ prefer native handling.

Fix. `respect.unordered.factors = "order"` for `ranger`, `categorical_feature` for LightGBM, or CatBoost's ordered target statistics.

7.8 Class imbalance defaults to predicting the majority

Trees split to minimize impurity; on imbalanced data the impurity-minimizing leaf prediction is the majority class. The default classifier predicts the majority for every row, achieving high accuracy and zero recall on the minority.

```
1 library(rpart); set.seed(1)
2 n <- 1000
3 df <- data.frame(x = rnorm(n),
4                 y = factor(sample(c(0, 1), n, replace = TRUE,
5                                 prob = c(0.95, 0.05))))
6 fit_raw <- rpart(y ~ x, data = df, method = "class")
7 fit_w    <- rpart(y ~ x, data = df, method = "class",
8                 parms = list(prior = c(0.5, 0.5)))
9 table(pred = predict(fit_raw, type = "class"), actual = df$y)
```

```
##      actual
```

```
## pred    0    1
##      0 946   54
##      1    0    0
```

```
1 table(pred = predict(fit_w,    type = "class"), actual = df$y)
```

```
##      actual
## pred    0    1
##      0 234    6
##      1 712   48
```

The unweighted tree predicts 0 for every row; the prior-weighted tree recovers a non-degenerate confusion matrix.

Diagnostic. Confusion matrix on the validation fold.

Fix. `parms = list(prior = ...)` for `rpart`, `class.weights` for `ranger`, `scale_pos_weight` for `xgboost`, or threshold tuning post hoc (Unit 3 §3.7).

7.9 Target-encoding leakage

Mean target encoding replaces a categorical level with the in-level mean of y , turning a K -level factor into one numeric column (§5.4). The naive implementation computes level means on the full dataset

and joins them to every row, so each test row's encoding contains its own y . CV scores look great; deployment collapses.

```
1 library(rpart); set.seed(1)
2 n <- 2000
3 df <- data.frame(city = factor(sample(paste0("c", 1:200), n, replace = TRUE)),
4                       y      = rnorm(n))           # city carries no signal
5
6 # Leaky: encode on the full dataset before splitting
7 df$city_te <- ave(df$y, df$city, FUN = mean)
8 sp <- sample(n, 0.8 * n)
9 trn <- df[sp, ]; tst <- df[-sp, ]
10 fit_leaky <- rpart(y ~ city_te, data = trn, control = rpart.control(cp = 0.005))
11 r2_leaky  <- 1 - mean((tst$y - predict(fit_leaky, tst))^2) / var(tst$y)
12
13 # Safe: encode using only training rows, apply the mapping to the test fold
14 enc <- tapply(trn$y, trn$city, mean)
15 trn$city_te <- enc[as.character(trn$city)]
16 tst$city_te <- enc[as.character(tst$city)]
17 tst$city_te[is.na(tst$city_te)] <- mean(trn$y)      # unseen levels: training mean
18 fit_safe <- rpart(y ~ city_te, data = trn, control = rpart.control(cp = 0.005))
```

```
19 r2_safe <- 1 - mean((tst$y - predict(fit_safe, tst))^2) / var(tst$y)
20
21 c(leaky = r2_leaky, safe = r2_safe)
```

```
##          leaky          safe
## 0.07285221 -0.08068258
```

The leaky encoding gives a noticeably positive test R^2 on a noise feature; the in-fold encoding returns near-zero (a slightly negative R^2 is expected from a noiseless null).

Diagnostic. Target-encode on the full dataset, then permute y and refit; any positive R^2 implicates the encoding.

Fix. Encode inside CV folds (`recipes::step_encode_mixed` from `embed`, or the training-fold join shown above), or use CatBoost's ordered target statistics (§5.4).